

Math 114C Notes

Neil Makur

January 2021

1 Partial Functions

Definition 1.1. A partial function p from X to Y is a function whose domain is a subset of X and whose codomain is Y . We write

$$p : X \rightharpoonup Y$$

to mean that p is a partial function from X to Y .

If $x \in \text{Dom}(p)$, we say that p is defined at x . We write $p(x) \downarrow$. We write $p(x) \downarrow = y$ for p is defined at x and that the value of p at x is y . We write $p(x) \uparrow$ to say that p is not defined at x . For two partial functions $f, g : X \rightharpoonup Y$ and $x \in X$, we say that $f(x) \simeq g(x)$ if either $f(x) \uparrow$ and $g(x) \uparrow$, or $f(x) \downarrow$, $g(x) \downarrow$, and $f(x) = g(x)$. If $f(x) \simeq g(x)$ for all $x \in X$, we write $f = g$. We also write $f \sqsubseteq g$ if, for all $x \in X$ with $f(x) \downarrow$, then $f(x) \simeq g(x)$. When $f \sqsubseteq g$, we say that g is an extension of f . The empty function ϵ is the unique partial function whose domain is the empty set.

If $f : X \rightharpoonup Y$ and $g : Y \rightharpoonup Z$ are partial functions, we define the partial function $g \circ f : X \rightharpoonup Z$ to have domain

$$\text{Dom}(g \circ f) = \{x \in X : f(x) \downarrow, g(f(x)) \downarrow\}$$

and be defined by $(g \circ f)(x) = g(f(x))$.

2 The Unlimited Register Machine

Our first model of computation will be the unlimited register machine, or URM. The URM contains infinitely many registers, labeled $R_1, R_2, \dots$. Each register R_i contains at any given moment $r_i \in \mathbb{N}$, which we call the contents of R_i . The machine is visualized as follows:

R_1	R_2	R_3	R_4	R_5	R_6	R_7	R_8	R_9	
r_1	r_2	r_3	r_4	r_5	r_6	r_7	r_8	r_9	$\dots$

However, for typographical reasons, we will often represent this instead as $(r_1, r_2, \dots)$.

The contents of the machine can be altered by the machine implementing certain instructions. There are four types of instructions: zero instructions, successor instructions, transfer instructions, and jump instructions. A URM-program is a finite list of instructions $I_1, I_2, \dots, I_n$, where each I_k is an instruction. We now discuss the types of instructions.

2.1 Zero Instructions

For each $n = 1, 2, \dots$, there is a zero instruction $Z(n)$. $Z(n)$ instructs the machine to change the contents of R_n to 0.

2.2 Successor Instructions

For each $n = 1, 2, \dots$, there is a successor instruction $S(n)$. $S(n)$ instructs the machine to add 1 to the contents of register R_n .

2.3 Transfer Instructions

For each $m, n \in \{1, 2, \dots\}$, there is a transfer instruction $T(m, n)$. $T(m, n)$ instructs the machine to replace the contents of register R_n with the contents of register R_m .

2.4 Jump Instructions

For each $m, n, q \in \{1, 2, \dots\}$, there is a jump instruction $J(m, n, q)$. The effect of $J(m, n, q)$ is conditional. If $r_m = r_n$, go to the q^{th} instruction in the program. Otherwise, when $r_m \neq r_n$, go to the next instruction.

Consider a program P with s instructions $I_1, \dots, I_s$. We say that a program P computation terminates or halts when the program instructs the machine to follow a nonexistent instruction (it tells the machine to execute I_t , where $t > s$). This can happen in two ways: The machine executes a jump instruction $J(m, n, q)$ when $r_m = r_n$ and $q > s$, or the machine executes I_s , and the next instruction is I_{s+1} . Note that a program does not need to halt. For example, the program given by $I_1 = S(1)$ and $I_2 = J(1, 1, 1)$.

Now consider the following program:

$$\begin{aligned} &J(1, 2, 6) \\ &S(2) \\ &S(3) \\ &J(1, 2, 6) \\ &J(1, 1, 2) \\ &T(3, 1) \end{aligned}$$

We apply this to the initial state $(6, 4, 0, 0, \dots)$. The program proceeds as follows:

$$I_1 : \quad (6, 4, 0, \dots) \rightarrow (6, 4, 0, \dots)$$

$$\begin{aligned}
I_2 : & \quad (6, 4, 0, \dots) \rightarrow (6, 5, 0, \dots) \\
I_3 : & \quad (6, 5, 0, \dots) \rightarrow (6, 5, 1, \dots) \\
I_4 : & \quad (6, 5, 1, \dots) \rightarrow (6, 5, 1, \dots) \\
I_5 : & \quad (6, 5, 1, \dots) \rightarrow (6, 5, 1, \dots) \\
I_2 : & \quad (6, 5, 1, \dots) \rightarrow (6, 6, 1, \dots) \\
I_3 : & \quad (6, 6, 1, \dots) \rightarrow (6, 6, 2, \dots) \\
I_4 : & \quad (6, 6, 2, \dots) \rightarrow (6, 6, 2, \dots) \\
I_6 : & \quad (6, 6, 2, \dots) \rightarrow (2, 6, 2, \dots)
\end{aligned}$$

3 URM-Computable Functions

Let P be a program and $a_1, a_2, \dots$ be an infinite sequence in $\mathbb{N}$. We write $P(a_1, a_2, \dots)$ for the computation under P with initial configuration $(a_1, a_2, \dots)$. We write $P(a_1, a_2, \dots) \downarrow$ to mean that the computation halts, and $P(a_1, a_2, \dots) \uparrow$ to mean that the computation does not halt. If $a_1, \dots, a_n$ is a finite sequence in $\mathbb{N}$, we write $P(a_1, \dots, a_n)$ for $P(a_1, \dots, a_n, 0, 0, \dots)$.

Let P be a program and n be a positive integer. We define a partial function $f_P^{(n)} : \mathbb{N}^n \rightarrow \mathbb{N}$ as follows. For $a_1, \dots, a_n \in \mathbb{N}$,

$$f_P^{(n)}(a_1, \dots, a_n) \downarrow \iff P(a_1, \dots, a_n) \downarrow$$

and

$$f_P^{(n)}(a_1, \dots, a_n) \downarrow = y \iff P(a_1, \dots, a_n) \downarrow \text{ and } y \text{ is the content of } R_1 \text{ when the computation halts.}$$

Definition 3.1. A partial function $g : \mathbb{N}^n \rightarrow \mathbb{N}$ is URM-computable if there is a program P such that $g = f_P^{(n)}$. We say that P computes g ,

For example, the total function $g(x, y) = x + y$ is URM-computable. The initial state is $(x, y, 0, 0, \dots)$. The program is given by:

$$\begin{aligned}
& J(2, 3, 5) \\
& S(1) \\
& S(3) \\
& J(1, 1, 1)
\end{aligned}$$

The total function

$$x - 1 = \begin{cases} 0 & x = 0 \\ x - 1 & x > 0 \end{cases}$$

is also URM-computable. The program takes in $(x, 0, 0, \dots)$ and is defined as follows.

$J(1, 2, 8)$
 $S(3)$
 $J(1, 3, 7)$
 $S(2)$
 $S(3)$
 $J(1, 1, 3)$
 $T(2, 1)$

The partial function

$$f(x) = \begin{cases} \frac{1}{2}x & x \equiv 0 \pmod{2} \\ \uparrow & x \equiv 1 \pmod{2} \end{cases}$$

Is URM-computable. It takes in $(x, 0, 0, \dots)$, and is defined as follows.

$J(1, 3, 6)$
 $S(2)$
 $S(3)$
 $S(3)$
 $J(1, 1, 1)$
 $T(2, 1)$

Note that two different programs P_1, P_2 can compute the same n -ary partial function. For example, let $P_1 = I_1, \dots, I_s$ be any program, and let $P_2 = P_1, J(1, 1, s + 2)$.

4 Decidable Problems

Let $R \subseteq \mathbb{N}^n$ be an n -ary relation on $\mathbb{N}$. Consider the following problem:

Given an n -tuple $(a_1, \dots, a_n)$, does R hold at $(a_1, \dots, a_n)$?

Definition 4.1. Let R be an n -ary relation function on $\mathbb{N}$. We say that R is decidable (or computable) if the characteristic function

$$\chi_R(a_1, \dots, a_n) = \begin{cases} 1 & R(a_1, \dots, a_n) \\ 0 & \neg R(a_1, \dots, a_n) \end{cases}$$

is a computable (total) function. When thinking of R as a set rather than a relation, we also call R a computable subset of $\mathbb{N}^n$.

For example, the relation $R \subseteq \mathbb{N}^2$ defined by $R(x, y) \iff x = y$ is decidable. Consider the following program taking in $(x, y, 0, 0, \dots)$.

$J(1, 2, 4)$
 $Z(1)$
 $J(1, 1, 6)$
 $Z(1)$
 $S(1)$

Another example is the relation $R(x) \iff x = 2$, which is defined by applying $S(2)$ twice and repeating the above program (changing the last entry of jump instructions).

5 Generating Computable Functions

Recall that the sum and products of continuous functions are continuous. We develop similar rules for computable functions.

Theorem 5.1. *The following functions are computable:*

- (a) *The zero function $\underline{0} : \mathbb{N} \rightarrow \mathbb{N}$ defined by $\underline{0}(x) = 0$ for all x .*
- (b) *The successor function $s : \mathbb{N} \rightarrow \mathbb{N}$ defined by $s(x) = x + 1$.*
- (c) *For each $n \geq 1$ and each $1 \leq i \leq n$, the projection function $U_i^n : \mathbb{N}^n \rightarrow \mathbb{N}$ defined by*

$$U_i^n(x_1, \dots, x_i, \dots, x_n) = x_i.$$

Proof. The programs for these are $Z(1)$, $S(1)$ and $T(i, 1)$ respectively. ■

Suppose that we wanted to execute program $P = I_1, \dots, I_s$, and then, if P halts, run $Q = I'_1, \dots, I'_t$. The natural idea is to run $I_1, \dots, I_s, I'_1, \dots, I'_t$, but this does not work.

Definition 5.2. A program $P = I_1, \dots, I_s$ is in standard form if every jump instruction occurring in P is of the form $J(m, n, q)$, where $q \leq s + 1$.

Lemma 5.3. *For every program P , there exists a program P^* in standard form such that for every $n \in \{1, 2, \dots\}$, we have $f_P^{(n)} = f_{P^*}^{(n)}$.*

Proof. Change every instruction in P of the form $J(m, n, q)$ with $q > s + 1$ to the instruction $J(m, n, s + 1)$. ■

Definition 5.4. Let P and Q be programs in standard form. The join or concatenation of P and Q is the program, denoted PQ , defined by $PQ = I_1, \dots, I_s, I_{s+1}, \dots, I_{s+t}$, where $P = I_1, \dots, I_s$ and $I_{s+1}, \dots, I_{s+t}$ are the instructions of Q with any instruction of the form $J(m, n, q)$ replaced with $J(m, n, s + q)$.

The join PQ is the program that first runs P , and then runs Q . Because we have developed a way to concatenate (standard form) programs, we will often list these programs as their own instructions.

Let P be a program. Since P is finite, there is a smallest $u \in \{1, 2, \dots\}$ such that no register R_v with $v > u$ is mentioned in P . In other words, if any of $Z(n)$, $S(n)$, $T(m, n)$ or $J(m, n, q)$ is in P , then $n, m \leq u$. We denote this number by $u = \rho(P)$. During a computation, the contents of any register R_v with $v > \rho(P)$ is not altered, and its contents do not affect those of R_n with $N \leq \rho(P)$. We call the registers $R_1, \dots, R_{\rho(P)}$ the working registers for P .

Let P be a program and let $f = f_P^{(n)}$. Suppose we are designing some larger program that will need to compute a value $f(x_1, \dots, x_n)$ as a subroutine, and suppose that $x_1, \dots, x_n$ can be found in registers $R_{\ell_1}, \dots, R_{\ell_n}$, and suppose we wish to store $f(x_1, \dots, x_n)$ in some R_ℓ . Finally, suppose that the working registers for P may be filled with unwanted information that we can clear before starting the subroutine. We use the notation $P[\ell_1, \dots, \ell_n \rightarrow \ell]$ to denote the program. The program is as follows

$$\begin{array}{c} T(\ell_1, 1) \\ \vdots \\ T(\ell_n, n) \\ Z(n+1) \\ \vdots \\ Z(\rho(P)) \\ P \\ T(1, \ell) \end{array}$$

Theorem 5.5. *Suppose that $f : \mathbb{N}^k \rightarrow \mathbb{N}$ and $g_1, \dots, g_k : \mathbb{N}^n \rightarrow \mathbb{N}$ are computable partial functions. Then the function $h : \mathbb{N}^n \rightarrow \mathbb{N}$ defined by*

$$h(x_1, \dots, x_n) \simeq f(g_1(x_1, \dots, x_n), \dots, g_k(x_1, \dots, x_n))$$

is computable.

Proof. Let $F, G_1, \dots, G_k$ be programs in standard form which compute $f, g_1, \dots, g_k$ respectively. Let $m = \max(n, k, \rho(F), \rho(G_1), \dots, \rho(G_k))$. Consider the following program:

$$\begin{array}{c} T(1, m+1) \\ \vdots \\ T(n, m+n) \\ G_1[m+1, \dots, m+n \rightarrow m+n+1] \\ \vdots \\ G_k[m+1, \dots, m+n \rightarrow m+n+k] \\ F[m+n+1, \dots, m+n+k \rightarrow 1] \end{array}$$

This computes h . ■

Corollary 5.6. *Suppose that $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is a computable partial function, and that $i_1, \dots, i_k$ is a sequence of indices from $1, \dots, n$, possible with repetitions. Then the partial function $h : \mathbb{N}^n \rightarrow \mathbb{N}$ defined by $h(x_1, \dots, x_n) \simeq f(x_{i_1}, \dots, x_{i_k})$ is computable.*

Proof. Note that $h \simeq f(U_{i_1}^n(x_1, \dots, x_n), \dots, U_{i_k}^n(x_1, \dots, x_n))$. Since the U_i^n are computable, h is computable. ■

For example, the function $g(x) = 2x$ is computable. We know that $h(x, y) = x + y$ is computable. Since $g(x) = h(x, x) = 2x$, it follows from the above corollary. Another example is that the function $f(x_1, x_2, x_3) = x_1 + x_2 + x_3$ is computable. We know that $h(x_1, x_2) = x_1 + x_2$ is computable. We also know that $f(x_1, x_2, x_3) = h(x_1 + x_2, x_3) = h(h(x_1, x_2), x_3)$. Thus f is computable, since it is obtained by substituting computable functions (we may use the projection functions U_i^j to isolate each variable).

Definition 5.7. A function $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ is defined by recursion (with parameters from $\mathbb{N}^k$) from $g : \mathbb{N}^k \rightarrow \mathbb{N}$ and a function $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ if f satisfies

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, y + 1) = h(\vec{x}, y, f(\vec{x}, y)) \end{cases}$$

for every $\vec{x} = (x_1, \dots, x_k)$ and $y \in \mathbb{N}$.

For example, the function $f(x, y) = xy$ is defined by recursion from $g(x) = 0$ and $h(x, y, z) = z + y$. The function $f(x) = 2^x$ is also defined by recursion from $g = 1$ and $h(y, z) = 2z$ (note that the $\vec{x}$ term is omitted, because $k = 0$ in this case). We can generalize this by defining $f(x, y)$ with $g(x) = 1$ and $h(x, y, z) = xz$.

Theorem 5.8. *Let $g : \mathbb{N}^k \rightarrow \mathbb{N}$ and $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ be partial functions. Then there exists a unique partial function $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ satisfying the recursive equations*

$$\begin{cases} f(\vec{x}, 0) \simeq g(\vec{x}) \\ f(\vec{x}, y + 1) \simeq h(\vec{x}, y, f(\vec{x}, y)) \end{cases}$$

We must properly understand the domain here. We have that $f(\vec{x}, 0) \downarrow \iff g(\vec{x}) \downarrow$ and that for $y \in \mathbb{N}$, $f(\vec{x}, y + 1) \downarrow \iff f(\vec{x}, 0) \downarrow \& \dots \& f(\vec{x}, y) \downarrow \& h(\vec{x}, y, f(\vec{x}, y)) \downarrow$.

Theorem 5.9. *Let $g : \mathbb{N}^k \rightarrow \mathbb{N}$ and $h : \mathbb{N}^{k+2} \rightarrow \mathbb{N}$ be computable partial functions. Then the partial function $f : \mathbb{N}^{k+1} \rightarrow \mathbb{N}$ defined by recursion from g and h is computable.*

Proof. The initial state of $f(\vec{x}, y)$ is $(\vec{x}, y, 0, \dots)$. Let G be a program computing g , let H be

a program compute h , and let $m = \max(\rho(G), \rho(H), k + 2)$. We use the following program.

$$\begin{aligned}
&T(1, m + 1) \\
&\vdots \\
&T(k + 1, m + k + 1) \\
&G[m + 1, \dots, m + k \rightarrow m + k + 3] \\
&J(m + k + 1, m + k + 2, k + 7) \\
&H[m + 1, \dots, m + k, m + k + 2, m + k + 3 \rightarrow m + k + 3] \\
&S(m + k + 2) \\
&J(1, 1, k + 3) \\
&T(m + k + 3, 1)
\end{aligned}$$

(We are essentially using R_{m+k+2} as a counter, and going until it is equal to y) ■

Proposition 5.10. *The following are computable:*

- (1) *The addition function $x + y$.*
- (2) *The multiplication function x^y .*
- (3) *The exponential function x^y .*
- (4) *The function $x - 1$ (which is 0 if $x = 0$).*
- (5) *Truncated Subtraction $x \dot{-} y = \begin{cases} x - y & x > y \\ 0 & x \leq y \end{cases}$.*
- (6) *The function $\text{sg}(x) = \begin{cases} 0 & x = 0 \\ 1 & x \neq 0 \end{cases}$.*
- (7) *The function $\overline{\text{sg}}(x) = 1 - \text{sg}(x)$.*
- (8) *The distance function $|x - y|$.*
- (9) *The factorial function $x!$.*
- (10) *The function $\min(x, y)$.*
- (11) *The function $\max(x, y)$.*
- (12) *The function $\text{rm}(x, y)$ denoting the remainder when y is divided by x , with $\text{rm}(0, y) = y$.*
- (13) *The function $\text{qt}(x, y)$ denoting the quotient when y is divided by x .*
- (14) *The function $\text{div}(x, y) = \begin{cases} 1 & x|y \\ 0 & x \nmid y \end{cases}$.*

Proof.

- (1) This is defined by recursion from $g(x) = x$ and $h(x, y, z) = z + 1$.
- (2) This is defined by recursion from $g(x) = \underline{0}(x)$ and $h(x, y, z) = z + x$.
- (3) This is defined by recursion from $g(x) = \underline{0}(x) + 1$ and $h(x, y, z) = xz$.
- (4) This is defined by recursion from $g = 0$ (note that g has no inputs) and $h(x, z) = x$.
- (5) This is defined by recursion from $g(x) = x$ and $h(x, y, z) = z - 1$.
- (6) This is defined by recursion from $g = 0$ and $h(x, z) = 1$.
- (7) This is defined by substitution, $\overline{\text{sg}}(x) = 1 - \text{sg}(x)$.
- (8) This is defined by substitution, $|x - y| = (x - y) + (y - x)$.
- (9) This is defined by recursion from $g = 1$ and $h(x, z) = (x + 1)z$.
- (10) This is defined by substitution $x - (x - y)$.
- (11) This is defined by substitution $x + (y - x)$.
- (12) This is defined by recursion from $g(x) = 0$ and $h(x, y, z) = (z + 1) \text{sg}(|z + 1 - x|)$.
- (13) This is defined by recursion from $g(x) = 0$ and $h(x, y, z) = z + +\overline{\text{sg}}(\text{rm}(x, y + 1))$.
- (14) This is defined by substitution by $\text{div}(x, y) = \overline{\text{sg}}((x, y))$.

■

Corollary 5.11. *Suppose that $f_1, \dots, f_k : \mathbb{N}^n \rightarrow \mathbb{N}$ are computable total functions, and let $M_1, \dots, M_k$ be decidable n -ary relations such that for each $x \in \mathbb{N}^n$, exactly one holds. Then the function $g : \mathbb{N}^n \rightarrow \mathbb{N}$ defined by*

$$g(x) = \begin{cases} f_1(x) & M_1(x) \\ f_2(x) & M_2(x) \\ \vdots & \vdots \\ f_k(x) & M_k(x) \end{cases}$$

is also computable.

Proof. Let $X_{M_1}, \dots, X_{M_k}$ be the computable characteristic functions. Then,

$$g(x) = X_{M_1}(x)f_1(x) + X_{M_2}(x)f_2(x) + \dots + X_{M_k}(x)f_k(x),$$

meaning that g is computable, since it is defined by substitution from computable functions. ■

Definition 5.12. Let $R, Q \subseteq \mathbb{N}^n$ be n -ary relations. We define

$$\begin{aligned}\neg R(x_1, \dots, x_n) &\iff (x_1, \dots, x_n) \notin R \\ (R \& Q)(x_1, \dots, x_n) &\iff R(x_1, \dots, x_n) \& Q(x_1, \dots, x_n) \\ (R \vee Q)(x_1, \dots, x_n) &\iff R(x_1, \dots, x_n) \vee Q(x_1, \dots, x_n)\end{aligned}$$

Corollary 5.13. Let R and Q be decidable n -ary relations. Then $\neg R$, $R \& Q$ and $R \vee Q$ are all decidable.

Proof. Let X_R and X_Q be the characteristic functions of R and Q . Then the characteristic function of $\neg R$ is $1 - X_R(x)$. The characteristic function of $R \& Q$ is $X_R(x) \cdot X_Q(x)$. Finally, the characteristic function of $R \vee Q$ is $\max(X_R(x), X_Q(x))$. ■

Theorem 5.14. Suppose that $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ is a computable total function. Then the functions

$$g(\vec{x}, y) = \sum_{z < y} f(\vec{x}, z) \quad h(\vec{x}, y) = \prod_{z < y} f(\vec{x}, z)$$

are also computable total functions.

Proof. Clearly both g and h can be defined by recursion from computable functions. ■

Corollary 5.15. Suppose that $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ and $k : \mathbb{N}^{n+m} \rightarrow \mathbb{N}$ are computable total functions. Then

$$g_1(\vec{x}, \vec{w}) = \sum_{z < k(\vec{x}, \vec{w})} f(\vec{x}z) \quad h_1(\vec{x}, \vec{w}) = \prod_{z < k(\vec{x}, \vec{w})} f(\vec{x}z)$$

are also computable total functions.

Proof. These are clearly substitutions of computable functions. ■

6 Minimalisation Operators

We write $\mu z(\dots)$ to denote the least $z \in \mathbb{N}$ such that $\dots$. Note that such a z may not exist. Now let $f : \mathbb{N}^{n+1} \rightharpoonup \mathbb{N}$ be a partial function. We define a new partial function $g : \mathbb{N}^n \rightharpoonup \mathbb{N}$ by

$$f(\vec{x}) = \mu y[f(\vec{x}, y) \downarrow = 0] = \begin{cases} y & f(\vec{x}, z) \downarrow \neq 0 \forall z < y, f(\vec{x}, y) \downarrow = 0 \\ \uparrow & \text{else} \end{cases}.$$

If R is an $n + 1$ -ary relation, then we also define

$$h(\vec{x}) = \mu y[R(\vec{x}, y)].$$

Theorem 6.1. *If $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ is a computable partial function, then the function*

$$g(\vec{x}) = \mu y[f(\vec{x}, y) = 0]$$

is also computable.

Proof. We use the following process: Move $\vec{x}$ to $R_{m+1}, \dots, R_{m+n}$ for large enough m . Compute $f(\vec{x}, k)$, where k is the content of R_{m+n+1} . If this is zero, place k in R_1 . If not, add one to k , and repeat.

Let F be a program computing f , and let $m = \max(\rho(F), n + 1)$. We use the following program

$$\begin{aligned} &T(1, m + 1) \\ &\vdots \\ &T(n, m + n) \\ &F[m + 1, \dots, m + n, m + n + 1 \rightarrow 1] \\ &J(1, m + n + 2, n + 5) \\ &S(m + n + 1) \\ &J(1, 1, n + 1) \\ &T(m + n + 1, 1). \end{aligned}$$

■

Corollary 6.2. *If R is an $n + 1$ -ary decidable relation, then the function $h(\vec{x}) = \mu y[R(\vec{x}, y)]$ is also computable.*

Proof. If $\mathcal{R}$ is a program computing R , then we can compute $\mu y[1 - \mathcal{R}(\vec{x}, y) = 0]$ by the theorem, which is what we need. ■

Example 6.3. The partial function $g : \mathbb{N} \rightarrow \mathbb{N}$ defined by

$$g(x) = \begin{cases} \sqrt{x} & x \text{ is a square} \\ \uparrow & \text{else} \end{cases}$$

is computable. To see this, notice that $h(x, y) = |x - y|$ is computable, and so by substitution, $h_1(x, y) = |x - y^2|$ is computable. Then $g(x) = \mu y[|x - y^2| = 0]$.

Before continuing, we introduce the following. If $f : \mathbb{N} \rightarrow \mathbb{N}$ is a total computable function, we define the graph relation of f to be the binary relation given by

$$G_f(x, y) \iff f(x) = y.$$

This is decidable, as the characteristic function is $\overline{\text{sg}}(|f(x) - y|)$.

Now let $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ be a computable partial function. Define

$$g(\vec{x}, y) = \mu z[(f(\vec{x}, z) = 0 \& z < y) \vee (z = y)] = \mu z < y[f(\vec{x}, z) = 0]$$

to be the bounded minimalization function.

Proposition 6.4. *Let $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ be a computable total function. Then, the bounded minimalisation function*

$$g(\vec{x}, y) = \mu z < y [f(\vec{x}, z) = 0]$$

is a computable total function.

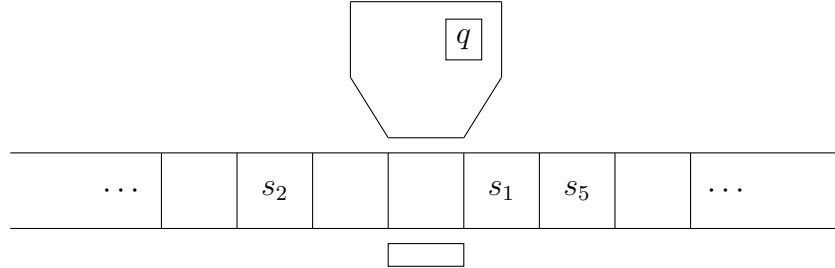
Proof. Since f is a total computable function, its graph relation $G_f(\vec{x}, z, w)$ (defined predictably) is decidable. Now, $f(\vec{x}, z) = 0$ if and only if $G_f(\vec{x}, z, 0)$, and so the relation $f(\vec{x}, z) = 0$ is decidable (we simply substituted $w = 0$). Since decidable relations are closed under $\&$ and $\vee$, the relation $R(\vec{x}, y, z) = ((z < y) \& (f(\vec{x}, z) = 0)) \vee (z = y)$ is decidable ($z < y$ has characteristic function $\text{sg}(y - z)$, and $z = y$ has characteristic function $\overline{\text{sg}}(|z - y|)$). Thus, $g(\vec{x}, y) = \mu z [R(\vec{x}, y, z)]$ is computable. ■

Thus, if R is a decidable $n + 1$ -ary relation, we can compute $f(\vec{x}, y) = \mu z < y [R(\vec{x}, z)]$. Note also we can compute the relation $\exists^< R$ defined by $(\exists^< R)(\vec{x}, y) \iff (\exists z < y) R(\vec{x}, z)$ by computing $\mu z < y [R(\vec{x}, z)]$ and checking that this is not equal to y (note that $R'(x, y) \iff x = y$ is computable, as $1 - (\max(x, y) - \min(x, y))$ works). We can also compute $\forall^< R$ defined by $(\forall^< R)(\vec{x}, y) \iff (\forall z < y) R(\vec{x}, z)$ as $\neg((\exists^< (\neg R))(\vec{x}, y))$.

7 Other Models of Computation

7.1 Turing Machines

A Turing machine M is composed of several parts, and can be visualised as follows.



The tape is infinite in both directions, and is segmented into infinitely many squares. Each square can be blank or contain a symbol from a fixed finite collection of symbols $\{s_1, \dots, s_n\}$ called the alphabet of the machine. We may denote being blank by the symbol B . The machine also has a reading head, which at any given time is reading a particular square. Also, at any given time, the head is in state q , where there are finitely many possible states $\{q_0, \dots, q_m\}$.

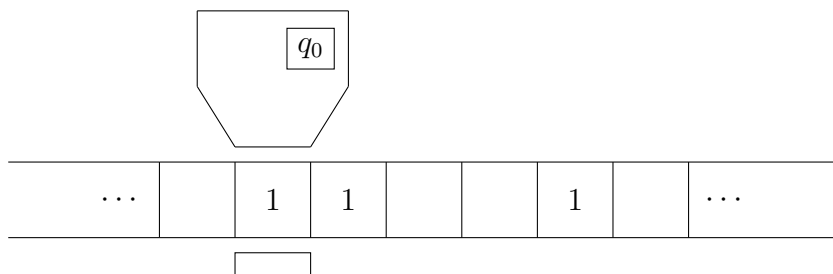
We now discuss how to program a Turing machine. There are three possible operations. If the machine is in state q_i and reading s_j (which may be blank), then the machine can erase s_j , replace it with s_k (which may be blank or equal to s_j), and change to state q_ℓ (which may be equal to q_i). This is abbreviated by $q_i s_j s_k q_\ell$. If the machine is in state q_i and reading s_j , then the head can move one square to the right and change to state q_ℓ (which may be equal to q_i). This is abbreviated by $q_i s_j R q_\ell$. Finally, if the machine is in state q_i and reading s_j , then the head can move one square to the left and change to state q_ℓ (which may be equal to q_i). This is abbreviated by $q_i s_j L q_\ell$.

Definition 7.1. A Turing program is a finite list of instructions such that, for any pair $q_i s_j$ (where q_i is a state and s_j is a symbol), there is at most one instruction of the form $q_i s_j \alpha q_\ell$.

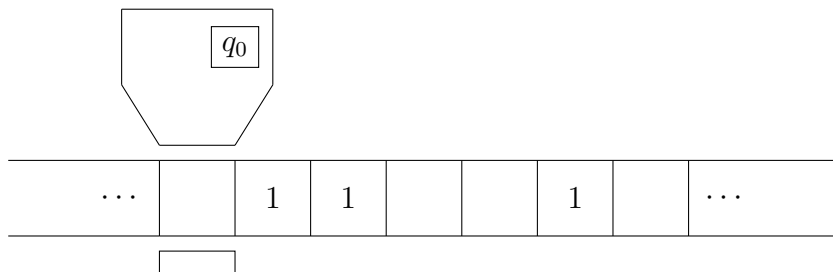
Note that these instructions do not need to be ordered. Given a program, we compute it in the obvious way. For example, consider a Turing machine with alphabet $\{1, B\}$, and let P be the program

$$\begin{aligned} q_0 1 L q_0 \\ q_0 B 1 q_1 \\ q_1 1 R q_2. \end{aligned}$$

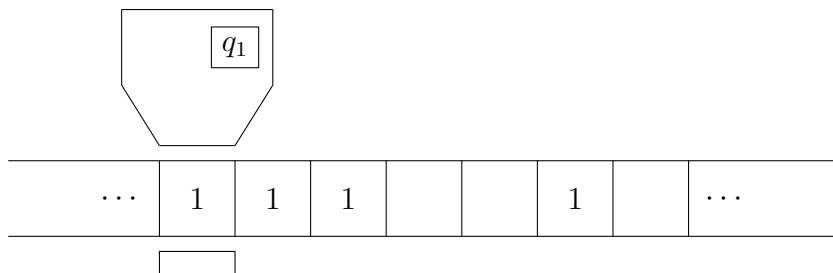
We compute with the initial state



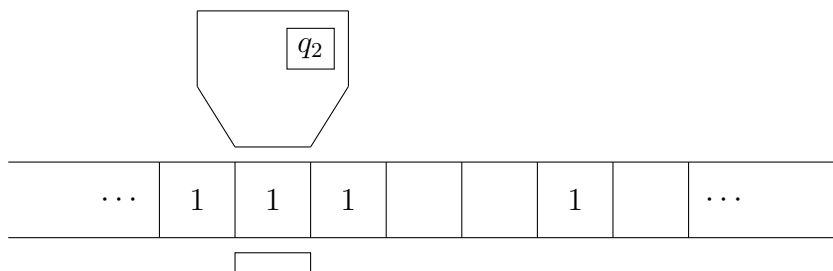
We first execute $q_0 1 L q_0$, which gives us



Now we execute $q_0 B 1 q_1$, which gives us



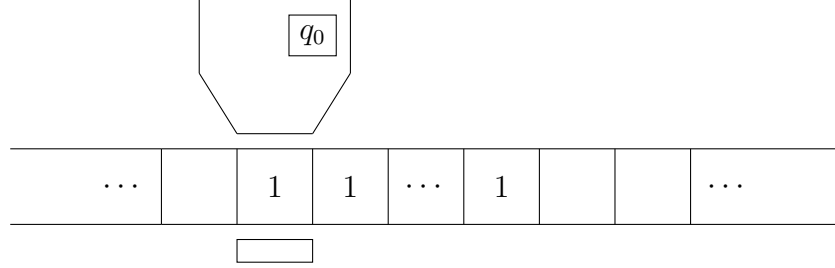
Executing $q_1 1 R q_2$, we get



Since there is no program starting with q_21 , we halt.

Definition 7.2. If the Turing machine is running a program P , in state q_i , and reading symbol s_j such that there is no instruction of the form $q_i s_j \alpha q_\ell$ in P , then we say the machine is in a halting state.

Suppose that the symbol s_1 is 1. Let P be a program. The partial function $f : \mathbb{N} \rightarrow \mathbb{N}$ computed by P is defined as follows: To compute $f(x)$, start with the initial state



where there are $x + 1$ many ‘1’s. Run the program P , and if the machine halts on this input, then

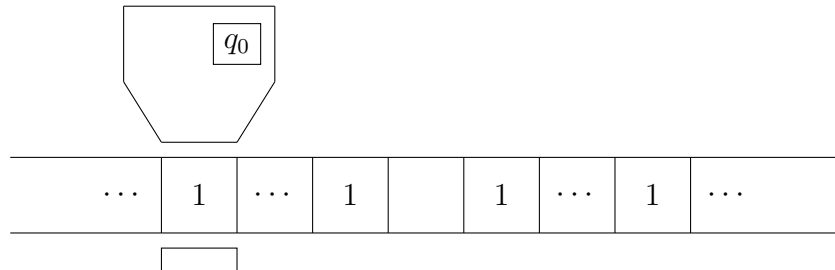
$f(x) \downarrow =$ the number of ‘1’s on the tape in the halting state

Definition 7.3. We say that a partial function $f : \mathbb{N} \rightarrow \mathbb{N}$ is Turing computable if there is a Turing machine and a program which computes it.

For example, $s(x) = x + 1$ is Turing computable with an empty program, and $f(x) = x + 2$ is Turing computable with

$$\begin{aligned} q_0 1 L q_0 \\ q_0 B 1 q_1 \\ q_1 1 R q_2. \end{aligned}$$

We can generalize this to a partial function taking in n inputs. For example, a function $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ on input $(x, y) \in \mathbb{N}^2$ starts the machine in state



where there are $x + 1$ ‘1’s before the blank, and $y + 1$ ‘1’s after the blank. For example, we can compute $f(x, y) = x + y$ with

$$\begin{aligned} q_0 1 B q_1 \\ q_1 B R q_1 \\ q_1 1 B q_2 \end{aligned}$$

7.2 Recursive Partial Functions

Definition 7.4. The class $\mathcal{R}$ of recursive partial functions is the smallest class of partial functions which contains the basic functions $\underline{0}$, $x + 1$, U_i^n , and is closed under the operations of substitution, recursion, and minimalisation.

An equivalent definition is a recursive partial function derivation, which is a finite sequence of partial functions

$$f_1, f_2, \dots, f_i, \dots, f_n,$$

where each $f_i : \mathbb{N}^{n_i} \rightarrow \mathbb{N}$ takes in n_i inputs such that, for each $1 \leq i \leq n$, either f_i is $\underline{0}$, $x + 1$, U_i^n , f_i is defined by substitution from $f_{j_1}, \dots, f_{j_m}$ with $j_1, \dots, j_m < i$, f_i is defined by recursion from $f_{j_1}, \dots, f_{j_m}$ with $j_1, \dots, j_m < i$, or f_i is defined by minimalisation from $f_{j_1}, \dots, f_{j_m}$ with $j_1, \dots, j_m < i$, and $f_n = f$.

For example, $h(x, y, z) = z + 1$ is defined by substitution as $h = s(U_1^3(x, y, z))$, so a recursive derivation of h is

$$s, U_1^3, h.$$

Now, $f(x, y) = x + y$ is defined by recursion from $f(x, 0) = x = U_1^1(x)$ and $f(x, y + 1) = h(x, y, f(x, y)) = f(x, y) + 1$. Thus, a recursive derivation of f is

$$s, U_1^3, h, U_1^1, f.$$

8 The Church-Turing Thesis

We will denote the set of URM-computable partial functions $\mathcal{C}$, the set of Turing computable partial functions $\mathcal{TC}$, and the set of recursive partial functions $\mathcal{R}$.

Theorem 8.1. *A partial function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is URM-computable if and only if f is Turing computable if and only if f is recursive. In other words, $\mathcal{C} = \mathcal{TC} = \mathcal{R}$.*

Because of this, we will simply say that f is computable. We will prove the following.

Theorem 8.2. *A partial function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is recursive if and only if it is URM-computable. In other words, $\mathcal{R} = \mathcal{C}$.*

Note that we have already proven that $\mathcal{R} \subseteq \mathcal{C}$.

Proposition 8.3. *The following functions are recursive:*

$$x - 1 \quad x - y \quad x + y \quad xy \quad x^y \quad \min(x, y) \quad \max(x, y) \quad (x, y) \text{ qt}(x, y) \quad \chi < \quad \chi \leq \quad \div(x, y),$$

where $\chi <$ is the characteristic function of the ordering relation $<$, and similarly for $\leq$.

Proof. Recall the proofs that these are URM-computable. These all work for recursive functions as well. We only need to show that $\chi <$ and $\chi \leq$ are recursive. We have that

$$\chi < = 1 - (x - y)$$

and that

$$\chi \leq = 1 - (1 - (y - x)).$$

■

Definition 8.4. An n -ary relation R on $\mathbb{N}$ is recursive if its characteristic function χ_R is recursive.

Proposition 8.5. Let R and Q be recursive n -ary relations on $\mathbb{N}$. Then, the following are recursive

$$\neg R \quad R \vee Q \quad R \& Q \quad R \implies Q \quad \exists^< R \quad \exists^{\leq} R \quad \forall^< R \quad \forall^{\leq} R.$$

Once again, the proof is the same as for URM-decidable relations.

Proposition 8.6. If a partial function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ has a recursive graph relation

$$G_f(\vec{x}, y) \iff f(\vec{x}) = y,$$

then f is recursive.

Proof. We have that $f(\vec{x}) = \mu y[G_f(\vec{x}, y)]$. ■

Note that there is a recursive function $p : \mathbb{N} \rightarrow \mathbb{N}$ such that $p(0) < p(1) < \dots$ that enumerates all the primes. To see this, notice that ‘ x is prime’ is recursive, as it is the same as checking $(y = 0) \vee (y = 1) \vee (y \nmid x)$ for all $y < x$, which can be computed by recursion. We can then define p by recursion, as $p(z + 1) = \mu y[y > p(z), y \text{ is prime}]$. We write p_n for $p(n)$.

8.1 Coding Sequences of Numbers

Let $\mathbb{N}^*$ be the set of all finite sequences of natural numbers, and consider $\emptyset$ as the empty sequence to be an element of $\mathbb{N}^*$. We define a function $\langle \cdot \rangle : \mathbb{N}^* \rightarrow \mathbb{N}$ by $\langle \emptyset \rangle = 1$ and

$$\langle x_0, x_1, \dots, x_{n-1} \rangle = p_0^{1+x_0} p_1^{1+x_1} \dots p_{n-1}^{1+x_{n-1}}$$

and define

$$\text{Seq} = \langle \mathbb{N}^* \rangle.$$

Elements of Seq are called sequence codes. We write $\text{Seq}(u)$ when u is a sequence code.

Proposition 8.7. $\text{Seq} \subseteq \mathbb{N}$ is recursive.

Proof. We have that $\text{Seq}(u)$ if and only if for all $p, q \leq u$ with p, q being prime, $p < q$, and $q|u$, imply $p|u$. These are all computable by recursion. ■

Proposition 8.8. There is a recursive function $\text{lh} : \mathbb{N} \rightarrow \mathbb{N}$ such that $\text{lh}(u) = n$ if $\text{Seq}(u)$ and $u = \langle x_0, \dots, x_{n-1} \rangle$ for some $x_0, \dots, x_{n-1} \in \mathbb{N}$, and $\text{lh}(u) = 0$ if $\neg \text{Seq}(u)$.

Proof. We can simply show that the graph of lh is recursive. We have that

$$\text{lh}(u) = n \iff [(\neg \text{Seq}(u) \vee u = 1) \& n = 0] \vee [\text{Seq}(u) \& p_{n-1} | u \& p_n \nmid u].$$
■

Proposition 8.9. *There is a recursive function $\text{proj} : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that, for any $x_0, \dots, x_{n-1} \in \mathbb{N}$,*

$$\text{proj}(\langle x_0, x_1, \dots, x_{n-1} \rangle, i) = \begin{cases} x_i & i < n \\ 0 & \text{else} \end{cases}$$

and $\text{proj}(u, i) = 0$ when $u \notin \text{Seq}$.

Proof. We have that

$$\text{proj}(u, i) = x \iff [(\neg \text{Seq}(u) \vee i > \text{lh}(u)) \& x = 0] \vee [\text{Seq}(u) \& p_i^{x+1} | u \& p_u^{x+2} \nmid u].$$

■

We write $(u)_i$ for $\text{proj}(u, i)$. We define

$$(u)_{i,j} = ((u)_i)_j = \text{proj}(\text{proj}(u, i), j),$$

and continue this.

Proposition 8.10. *There is a recursive function $\text{append} : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that*

$$\text{append}(\langle x_0, x_1, \dots, x_{n-1} \rangle, y) = \langle x_0, x_1, \dots, x_{n-1}, y \rangle.$$

Moreover, there are recursive functions $\text{res} : \mathbb{N}^2 \rightarrow \mathbb{N}$ and $*$: $\mathbb{N}^2 \rightarrow \mathbb{N}$ such that, for any $x_0, x_1, \dots, x_{n-1}, y_0, y_1, \dots, y_{m-1} \in \mathbb{N}$, we have

$$\text{res}(\langle x_0, x_1, \dots, x_{n-1} \rangle, i) = \langle x_0, x_1, \dots, x_{i-1} \rangle \quad \text{for } i < n$$

and

$$\langle x_0, x_1, \dots, x_{n-1} \rangle * \langle y_0, y_1, \dots, y_{m-1} \rangle = \langle x_0, x_1, \dots, x_{n-1}, y_0, y_1, \dots, y_{m-1} \rangle,$$

and $u * v = 0$ if either $u \notin \text{Seq}$ or $v \notin \text{Seq}$.

Proof. We have that

$$\text{append}(u, y) = u \cdot p_{\text{lh}(u)}^{y+1}.$$

Now, define $h(u, v, 0) = u$ and $h(u, v, i+1) = \text{append}(h(u, v, i), (v)_i)$. Then

$$h(\langle x_0, \dots, x_{n-1} \rangle, \langle y_0, \dots, y_{m-1} \rangle, i) = \langle x_0, \dots, x_{n-1}, y_0, \dots, y_{i-1} \rangle,$$

so $\text{res}(v, i) = h(\langle \emptyset \rangle, y, i)$. We also have that $u * v = h(u, v, \text{lh}(v))$. ■

8.2 URM-Computable Functions are Recursive

We now start indexing at 0, so that our registers are $R_0, R_1, \dots$, our programs are $P = I_0, I_1, \dots, I_{s-1}$, and when a register machine halts, the output register is now R_0 . We use $\#*$ to denote the code of $*$. We now assign numbers to the different types of instructions:

$$\#Z = 0 \quad \#S = 1 \quad \#T = 2 \quad \#J = 3,$$

and we assign codes for instructions

$$\begin{aligned}\#Z(n) &= \langle \#Z, n \rangle \\ \#S(n) &= \langle \#S, n \rangle \\ \#T(n, m) &= \langle \#T, n, m \rangle \\ \#J(n, m, q) &= \langle \#J, n, m, q \rangle.\end{aligned}$$

Having done this, given a program $P = I_0, \dots, I_{s-1}$, we assign

$$\#P = \langle \#I_0, \dots, \#I_{s-1} \rangle.$$

Note that the set of instruction codes (IC) and the set of program codes (PC) are recursive.

Note also that there is a function $\hat{\rho} : \mathbb{N} \rightarrow \mathbb{N}$ such that, for any program P , $\hat{\rho}(\#P) = \rho(P)$ (decoding $\#P$ and taking the maximum over all appearing registers does this).

Definition 8.11. Let $P = I_0, \dots, I_{s-1}$, let n be a positive integer, and let $m = \max(\rho(P), n-1)$. A code u is a state code for $f_P^{(n)}$ if either there is $t < s$ such that $u = \langle t, \#I_t, \langle r_0, \dots, r_m \rangle \rangle$ for some $r_0, \dots, r_m \in \mathbb{N}$, or there is $t \geq s$ such that $u = \langle t, 0, \langle r_0, \dots, r_m \rangle \rangle$.

We can imagine this being in a certain state: the content of our registers is $r_0, \dots, r_m$, and I_t is the next instruction to be computed (and 0 if we have reached the end of the program). We define

$$\text{InsNum}(u) = (u)_0 \quad \text{Ins}(u) = (u)_1 \quad \text{Reg}(u, i) = (u)_{2,i}.$$

These state, the index of the next instruction, the next instruction, and the content of register i respectively.

Lemma 8.12. *There is a recursive relation $\text{StateCode} \subseteq \mathbb{N}^3$ such that*

$$\text{StateCode}(n, e, u) \iff e \text{ is the code of program } P \text{ and } u \text{ is a state code for } f_P^{(n)}.$$

Proof. We use the following equivalence:

$$\begin{aligned}\text{StateCode}(n, e, u) \iff & n > 0 \& PC(e) \& \text{Seq}(u) \& \text{lh}(u) = 3 \& \text{Seq}((u)_2) \\ & \& \text{lh}((u)_2) = \max(\hat{\rho}(e), n) \& \text{Ins}(u) = (e)_{\text{InsNum}(u)}\end{aligned}$$

■

We also define

$$\text{HaltState}(n, e, u) \iff \text{StateCode}(n, e, u) \& \text{Ins}(u) = 0,$$

which holds if and only if u is a halting state code for $f_P^{(n)}$, where P is coded by e .

Lemma 8.13. *There is a recursive relation CmpSt such that*

$$\begin{aligned}\text{CmpSt}(n, e, u, v) \iff & e \text{ is a code for program } P, u, v \text{ are state codes for } f_P^{(n)}, \\ & v \text{ is the code for the state that results when applying} \\ & \text{the instruction of } u \text{ to the contents of } u.\end{aligned}$$

Proof. Let $\ell = \text{lh}((u_2))$. We have that

$$\begin{aligned}
& \text{CmpSt}(n, e, u, v) \iff \text{StateCode}(n, e, u) \& \neg \text{HaltState}(n, e, u) \\
& \quad \& \text{StateCode}(n, e, v) \& \exists j, k, q \leq u \text{ such that} \\
& \quad \left[\left(\text{Ins}(u) = \#Z(j) \rightarrow \text{Reg}(v, j) = 0 \right. \right. \\
& \quad \quad \& (\forall i < \ell)(i \neq j \rightarrow \text{Reg}(u, i) = \text{Reg}(v, i)) \\
& \quad \quad \& \text{InsNum}(v) = \text{InsNum}(u) + 1 \Big) \\
& \quad \& \left(\text{Ins}(u) = \#S(j) \rightarrow \text{Reg}(v, j) = \text{Reg}(u, j) + 1 \right. \\
& \quad \quad \& (\forall i < \ell)(i \neq j \rightarrow \text{Reg}(u, i) = \text{Reg}(v, i)) \\
& \quad \quad \& \text{InsNum}(v) = \text{InsNum}(u) + 1 \Big) \\
& \quad \& \left(\text{Ins}(u) = \#T(j, k) \rightarrow \text{Reg}(v, k) = \text{Reg}(u, j) \right. \\
& \quad \quad \& (\forall i < \ell)(i \neq k \rightarrow \text{Reg}(u, i) = \text{Reg}(v, i)) \\
& \quad \quad \& \text{InsNum}(v) = \text{InsNum}(u) + 1 \Big) \\
& \quad \& \left(\text{Ins}(u) = \#J(j, k, q) \rightarrow (\forall i < \ell) \text{Reg}(u, i) = \text{Reg}(v, i) \right. \\
& \quad \quad \& \left\{ (\text{Reg}(u, j) = \text{Reg}(u, k) \& \text{InsNum}(v) = q) \right. \\
& \quad \quad \quad \vee (\text{Reg}(u, j) \neq \text{Reg}(u, k)) \\
& \quad \quad \quad \& \left. \text{InsNum}(v) = \text{InsNum}(u) + 1 \right\} \Big) \Big]
\end{aligned}$$

■

Theorem 8.14. *For each positive integer n , there is a recursive $(n+2)$ -ary relation $T_n(e, \vec{x}, y)$ and a recursive function $U : \mathbb{N} \rightarrow \mathbb{N}$ such that, for any program P with $e = \#P$ and every $\vec{x} \in \mathbb{N}^n$,*

$$f_P^{(n)}(\vec{x}) = U(\mu y T_n(e, \vec{x}, y)).$$

Proof. Recall that $\text{HaltState}(n, e, u)$ holds if and only if u is a halting code for $f_P^{(n)}$, where P is the program coded by e . We define a recursive $(n+2)$ -ary relation

$$\begin{aligned}
& \text{InitialState}_n(e, u, \vec{x}) \iff \text{StateCode}(n, e, u) \& \text{InsNum}(u) = 0 \& \text{Reg}(u, 0) = x_0 \\
& \quad \& \cdots \& \text{Reg}(u, n-1) = x_{n-1} \& (\forall i < \text{lh}((u)_2)[i \geq n \rightarrow \text{Reg}(u, i) = 0]
\end{aligned}$$

which holds if and only if u is a state code for the initial configuration of the machine which computes $f_P^{(n)}$ with input $\vec{x}$. Now, we define

$$\begin{aligned}
& T_n(e, \vec{x}, y) \iff PC(e) \& \text{Seq}(y) \& (\forall i < \text{lh}(y) - 2)[\text{CmpSt}(n, e, (y)_i, (y)_{i+1}) \\
& \quad \& \text{InitialState}_n(e, (y)_0, \vec{x}) \& \text{HaltState}(n, e, (y)_{\text{lh}(y)-1})
\end{aligned}$$

and

$$U(y) = (y)_{\text{lh}(y)-1,2,0}.$$

■

Note this proves that URM-Computable functions are recursive, as $f_P^{(n)}$ is an arbitrary URM-Computable function, which is therefore recursive, as T_n and U are recursive. We call T_n the Kleene T -predicate.

8.3 Effective Enumeration

Fix $n \in \mathbb{Z}^+$. For each $e \in \mathbb{N}$, define

$$\varphi_e^{(n)} : \mathbb{N}^n \rightarrow \mathbb{N}, \quad \varphi_e^{(n)}(\vec{x}) = U(\mu y T_n(e, \vec{x}, y)).$$

If e is the code of program P , then $\varphi_e^{(n)} = f_P^{(n)}$. If e is not the code of a program, then

$$\varphi_e^{(n)} = \varepsilon,$$

where ε denoted the (unique) computable partial function with empty domain. In other words, $\varphi_e^{(n)}(\vec{x}) \uparrow$ for all $\vec{x} \in \mathbb{N}^n$. For each $n, e \in \mathbb{N}$, we define

$$W_e^{(n)} = \text{Dom}(\varphi_e^{(n)}) = \{\vec{x} \in \mathbb{N}^n : \varphi_e^{(n)}(\vec{x}) \downarrow\}$$

and

$$E_e^{(n)} = \text{Ran}(\varphi_e^{(n)}) = \{y \in \mathbb{N} : \exists \vec{x} \in \mathbb{N}^n, \varphi_e^{(n)}(\vec{x}) \downarrow = y\}.$$

If $n = 1$, we omit the superscript (1).

Theorem 8.15. *Let n be a positive integer. There is a program P such that*

$$f_P^{(n+1)}(e, \vec{x}) \simeq \varphi_e^{(n)}(\vec{x})$$

Proof. Let

$$g(e, \vec{x}) = U(\mu y T_n(e, \vec{x}, y)).$$

Thus, g is recursive, and so URM-computable. Take a program P so that $g(e, \vec{x}) = f_P^{(n+1)}(e, \vec{x})$. This satisfies the conditions of the theorem. ■

8.4 Diagonalization

We now discuss diagonalization. As an example, we will prove there is no sequence functions $f_n : \mathbb{N} \rightarrow \{0, 1\}$ which contains every possible function from $\mathbb{N}$ to $\{0, 1\}$. Let $f_0, f_1, f_2, \dots$ be a sequence of functions $f_n : \mathbb{N} \rightarrow \{0, 1\}$. Consider an array

$$\begin{array}{cccc} f_0(0) & f_0(1) & f_0(2) & \cdots \\ f_1(0) & f_1(1) & f_1(2) & \cdots \\ f_2(0) & f_2(1) & f_2(2) & \cdots \\ \vdots & \vdots & \vdots & \ddots \end{array}$$

Define $g : \mathbb{N} \rightarrow \{0, 1\}$ by $g(n) = 1 - f_n(n)$. Then $g(x) \neq f_n(x)$, because $g(n) = 1 - f_n(n) \neq f_n(n)$. Thus, $g(x)$ is not listed in $f_0, f_1, \dots$

Theorem 8.16. *There is a function $f : \mathbb{N} \rightarrow \mathbb{N}$ that is not computable.*

Proof. We define f so that $f \neq \varphi_e^{(n)}$ for all $e \in \mathbb{N}$. It follows that f is not computable. We pick $f(n) \neq \varphi_n(n)$. We define

$$f(n) = \begin{cases} \varphi_n(n) + 1 & \varphi_n(n) \downarrow \\ 0 & \text{else} \end{cases}.$$

Then f is total. Now suppose that $f = \varphi_e$. Since f is total, φ_e is total, so $\varphi_e(e) \downarrow$. Thus, $\varphi_e(e) + 1 = f(e) = \varphi_e(e)$, a contradiction. ■

Definition 8.17. The (unitary, diagonal) halting problem is the relation $H \subseteq \mathbb{N}$ defined by

$$H(e) \iff \varphi_e(e) \downarrow.$$

Theorem 8.18. *The halting problem is undecidable.*

Proof. We showed that

$$f(n) = \begin{cases} \varphi_n(n) + 1 & H(n) \\ 0 & \neg H(n) \end{cases}$$

is not computable. However, if $H(n)$ was computable, then f would be defined by cases from computable functions and computable cases, and so computable. Since f is not computable, H cannot be computable. ■

8.5 The s-m-n Theorem

Let $f : \mathbb{N} \rightarrow \mathbb{N}$ be a computable partial function. Fix $a \in \mathbb{N}$ and define $g_a : \mathbb{N} \rightarrow \mathbb{N}$ by

$$g_a(y) \simeq f(a, y).$$

We want to determine if there is a program which, given a , can compute a program code for g_a .

Lemma 8.19. *There is a computable total function $\sigma : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that*

$$\sigma(i, n) = \underbrace{\langle \#S(i), \dots, \#S(i) \rangle}_{n \text{ times}}.$$

Proof. Note that σ is defined by recursion from computable functions

$$\begin{aligned} \sigma(i, 0) &= 1 \\ \sigma(i, n+1) &= \text{append}(\sigma(i, n), \#S(i)). \end{aligned}$$

■

Lemma 8.20. *There is a computable function $\tau : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that for any program P with $e = \#P$, $\tau(e, n)$ is the code for the program which is the same as P except that the indices for the jump instructions have all been increased by n .*

Proof. We have that

$$\begin{aligned} \tau(e, n) = u \iff & (\neg \text{Seq}(e) \& u = 0) \vee (\text{Seq}(e) \& (\forall i < \text{lh}(e)) \left\{ \left[\text{lh}((e)_i) \neq 4 \& (e)_i = (u)_i \right] \right. \\ & \left. \vee \left[\text{lh}((e)_i) = 4 \& (u)_{i,0} = (e)_{i,0} \& (u)_{i,1} = (e)_{i,1} \& (u)_{i,2} = (e)_{i,2} \& (u)_{i,3} = (e)_{i,3} + n \right] \right\}). \end{aligned}$$

■

Theorem 8.21. *Let $f : \mathbb{N}^2 \rightarrow \mathbb{N}$ be a computable partial function. There is a computable total function $k : \mathbb{N} \rightarrow \mathbb{N}$ such that, for every $x, y \in \mathbb{N}$,*

$$\varphi_{k(x)}(y) \simeq f(x, y).$$

Proof. Let P be a program computing f . For a particular x , the program coded by $k(x)$ should be

$$\begin{aligned} & T(0, 1) \\ & Z(0) \\ & S(0) \\ & \vdots \\ & S(0) \\ & P[0, 1 \rightarrow 0] \end{aligned}$$

where there are x many $S(0)$'s. We need to show that given x , we can compute $k(x)$. This follows, as the code would be

$$\langle \#T(0, 1), \#Z(0) \rangle * \sigma(0, x) * \tau(\#P, x + 2).$$

■

We also have a more generalized version.

Theorem 8.22 (The s-m-n Theorem). *For all $m, n \geq 1$, there is a computable function $s_n^m : \mathbb{N}^{m+1} \rightarrow \mathbb{N}$ such that*

$$\varphi_{s_n^m(e, \vec{x})}^{(n)}(\vec{y}) = \varphi_e^{(m+n)}(\vec{x}, \vec{y})$$

for all $e \in \mathbb{N}$ and $\vec{x} \in \mathbb{N}^m$ and $y \in \mathbb{N}^n$.

Proof. We have that $s_n^m(e, \vec{x})$ should be the code of

$$\begin{aligned}
& T(n-1, m+n-1) \\
& T(n-2, m+n-2) \\
& \vdots \\
& T(0, m) \\
& Z(0) \\
& Z(1) \\
& \vdots \\
& Z(m-1) \\
& S(0) \\
& \vdots \\
& S(0) \\
& \vdots \\
& S(m-1) \\
& \vdots \\
& S(m-1) \\
& P[0, 1, \dots, m+n-1 \rightarrow 0]
\end{aligned}$$

where $e = \#P$, and $S(k)$ appears x_k many times. Thus, we have that

$$s_n^m(e, \vec{x}) = \langle T(n-1, m+n-1), \dots, T(0, m) \rangle * \sigma(0, x_0) * \dots * \sigma(m-1, x_{m-1}) * \tau(e, m+n+x_0+\dots+x_{m-1}).$$

■

For example, there is a computable total unary function k such that $\varphi_{k(x)}(y) = x^y$ for all $x, y \in \mathbb{N}$, since if we define $f(x, y) = x^y$, then there is a k such that $\varphi_{k(x)}(y) = f(x, y) = x^y$. As another example, there is a computable total unary function s such that $W_{s(n)} = \text{Dom}(\varphi_{s(n)})$ is the set of all multiples of n . We can define

$$f(n, x) = \begin{cases} 1 & n|x \\ \uparrow & n \nmid x \end{cases}$$

This is computable, as it is defined by cases by a decidable relation. Thus, the s-m-n theorem implies that there is a computable function $s : \mathbb{N} \rightarrow \mathbb{N}$ such that $\varphi_{s(n)}(x) = f(n, x)$, so that $W_{s(n)}$ is the set of multiples of n .

We define an operation on computable partial functions to be an assignment taking some number of computable partial functions and outputting another computable partial function. For example, the addition operation is

$$f, g : \mathbb{N}^n \rightharpoonup \mathbb{N} \mapsto f + g : \mathbb{N}^n \rightharpoonup \mathbb{N}.$$

We say that this operation is effective in the codes because there is a computable function which, given codes for f and g , computes a code for $f+g$. In other words, there is $\sigma : \mathbb{N}^2 \rightarrow \mathbb{N}$ such that, for $f = \varphi_e$ and $g = \varphi_z$, then $\varphi_{\sigma(e,z)}(x) = \varphi_e(x) + \varphi_z(x)$. To see this, we define

$$h(e, z, x) = \varphi_e(x) + \varphi_z(x),$$

which is computable. By the s-m-n theorem, there is some σ such that

$$\varphi_{\sigma(e,x)}(x) = h(e, z, x) = \varphi_e(x) + \varphi_z(x).$$

8.6 The Second Recursion Theorem

Theorem 8.23 (The Second Recursion Theorem). *For every computable partial function $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$, there exists a z^* such that, for all $\vec{x} \in \mathbb{N}^n$*

$$\varphi_{z^*}^{(n)}(\vec{x}) = f(z^*, \vec{x}).$$

Proof. Let $s : \mathbb{N}^2 \rightarrow \mathbb{N}$ be from the s-m-n theorem satisfying

$$\varphi_{s(e,y)}^{(n)}(\vec{x}) = \varphi_e^{(n+1)}(y, \vec{x}).$$

Define $g(e, \vec{x}) = f(s(e, e), \vec{x})$. We have that g is computable by substitution. Pick e_0 so that $g = \varphi_{e_0}^{(n+1)}$. Define $z^* = s(e_0, e_0)$. For any $\vec{x} \in \mathbb{N}^n$, we have that

$$\varphi_{z^*}^{(n)}(\vec{x}) = \varphi_{s(e_0, e_0)}^{(n)}(\vec{x}) = \varphi_{e_0}^{(n+1)}(e_0, \vec{x}) = g(e_0, \vec{x}) = f(s(e_0, e_0), \vec{x}) = f(z^*, \vec{x}).$$

■

Corollary 8.24. *For every total recursive function $f : \mathbb{N} \rightarrow \mathbb{N}$ and positive integer n , there exists a $z^* \in \mathbb{N}$ such that $\varphi_{z^*}^{(n)} = \varphi_{f(z^*)}^{(n)}$.*

Proof. Define $g(z, \vec{x}) = \varphi_{f(z)}^{(n)}(\vec{x})$. Then there is some z^* so that $\varphi_{z^*}^{(n)}(\vec{x}) = g(z^*, \vec{x}) = \varphi_{f(z^*)}^{(n)}(\vec{x})$. ■

An application of this is that there exists z^* so that $\varphi_{z^*}(x) = z^*$ for all x . We can apply The Second Recursion Theorem to $f(z, x) = z$.

Definition 8.25. An index for a computable partial function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is a number e such that $\varphi_e^{(n)} = f$. A set $A \subseteq \mathbb{N}$ is an index set if, for all e_0, e_1 with $\varphi_{e_0} = \varphi_{e_1}$, then $e_0 \in A \iff e_1 \in A$.

For example, $\emptyset$ and $\mathbb{N}$ are index sets. As another example, $\{e \in \mathbb{N} : \varphi_e(e) \downarrow = 0\}$ is not an index set. To see this, use The Second Recursion Theorem to find z^* such that $\varphi_{z^*} = \begin{cases} 0 & x = z^* \\ \uparrow & \text{else} \end{cases}$ (apply this on $f(z, x) = \begin{cases} 0 & x = z \\ \uparrow & \text{else} \end{cases}$). Pick $e \neq z^*$ such that $\varphi_e = \varphi_{z^*}$. Then $z^* \in A$ but $e \notin A$.

Theorem 8.26 (Rice). *Let $A \neq \emptyset, \mathbb{N}$ be an index set. Then A is undecidable.*

Proof. Let $a \in A$ and $b \notin A$. Define

$$f(z, x) = \begin{cases} \varphi_a(x) & z \notin A \\ \varphi_b(x) & z \in A \end{cases}.$$

If A is decidable, then f is computable. Assume that A is decidable. By The Second Recursion Theorem, there is a z^* such that

$$\varphi_{z^*}(x) = \begin{cases} \varphi_a(x) & z^* \notin A \\ \varphi_b(x) & z^* \in A \end{cases}.$$

If $z^* \notin A$, then $\varphi_{z^*} = \varphi_a$, so $a \notin A$, which is a contradiction. If $z^* \in A$, then $\varphi_{z^*} = \varphi_b$, so $b \in A$, which is also a contradiction. Thus, A cannot be decidable. ■

For example, the index set $A = \{e \in \mathbb{N} : \varphi_e = 0\}$ is undecidable.

Definition 8.27. Let $R \subseteq \mathbb{N}^m$ and $Q \subseteq \mathbb{N}^n$ be relations. A reduction from R to Q is a collection of total functions $f_1, \dots, f_n : \mathbb{N}^m \rightarrow \mathbb{N}$ such that, for any $\vec{x} \in \mathbb{N}^m$,

$$R(\vec{x}) \iff Q(f_1(\vec{x}), \dots, f_n(\vec{x})).$$

If $f_1, \dots, f_n$ are computable, then we say that $(f_1, \dots, f_n)$ is a computable reduction from R to Q . We say that R can be computably reduced to Q if there is a computable reduction from R to Q .

For example, the halting problem can be computably reduced to the (non-diagonal) halting problem $R(x, y) \iff \varphi_x(y) \text{ halts}$, as

$$H(x) \iff \varphi_x(x) \text{ halts} \iff R(x, x),$$

so we may use the functions $f_1 = f_2 = U_1^2$.

Theorem 8.28. If the halting problem H can be computably reduced to a relation $R \subseteq \mathbb{N}^n$, then R is undecidable.

Proof. Let $f_1, \dots, f_n : \mathbb{N} \rightarrow \mathbb{N}$ be computable such that

$$H(x) \iff R(f_1(x), \dots, f_n(x)).$$

If R is decidable, then H is decidable, meaning that R must be undecidable. ■

Thus, the (non-diagonal) halting problem is undecidable. Similarly, if Q is an undecidable m -ary relation, and R is an n -ary relation where Q is computably reducible to R , then R is also undecidable.

Example 8.29. The relation

$$R(x, y) \iff \varphi_x = \varphi_y$$

is undecidable. To see this, consider $Z(x) \iff \varphi_x = \underline{0}$. Then Z is a nontrivial index set, and so undecidable. Fix e_0 with $\varphi_{e_0} = \text{fcn}$. Then we have a computable reduction

$$Z(x) \iff R(x, e_0).$$

Since Z is undecidable, R is undecidable.

As another example, the relation

$$R(e, y) \iff y \in \text{Ran}(\varphi_e)$$

is undecidable. Define

$$f(e, y) = \begin{cases} y & H(e) \\ \uparrow & \neg H(e) \end{cases}.$$

This is computable, since

$$f(e, y) = y + 0 \cdot \mu z[T_1(e, e, z)].$$

By the s-m-n theorem, there is a computable s such that $\varphi_{s(e)}(y) = f(e, y)$. Then,

$$H(e) \implies \text{Ran}(\varphi_{s(e)}) = \mathbb{N}$$

and

$$\neg H(e) \implies \text{Ran}(\varphi_{s(e)}) = \emptyset,$$

so

$$H(e) \iff 0 \in \text{Ran}(\varphi_{s(e)}) \iff R(s(e), 0).$$

9 Partially Decidable Relations

Recall the Keene T Predicate, which is a recursive relation $T_n(e, \vec{x}, y)$ and a recursive total function $U : \mathbb{N} \rightarrow \mathbb{N}$ such that $\varphi_e^{(n)} = U(\mu y T_n(e, \vec{x}, y))$. Recall that we define $W_e^{(n)} = \text{Dom}(\varphi_e^{(n)})$. Note that

$$\vec{x} \in W_e^{(n)} \iff U(\mu y T_n(e, \vec{x}, y)) \downarrow \iff \mu y T_n(e, \vec{x}, y) \downarrow \iff \exists y T_n(e, \vec{x}, y).$$

Fix e and consider the set $W_e = \{x \in \mathbb{N} : \varphi_e(x) \downarrow\}$.

Definition 9.1. An n -ary relation $R \subseteq \mathbb{N}^n$ is partially decidable if the partial function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ defined by

$$f(\vec{x}) = \begin{cases} 1 & R(\vec{x}) \\ \uparrow & \text{else} \end{cases}$$

is computable.

This function f is called the partial characteristic function of R , and any algorithm computing f is called a partial decision procedure for R . Partially decidable relations are also called semi-decidable, semi-computable and partially solvable.

The halting problem is partially decidable. To see this, note that the function

$$1 + 0 \cdot \mu y T_n(x, x, y)$$

is a partial decision procedure for the halting problem, where T_n is from the Kleene T Predicate. Note that any decidable relation is partially decidable, as if $R \subseteq \mathbb{N}^n$ is decidable, then

$$f(\vec{x}) = \begin{cases} 1 & R(\vec{x}) \\ \uparrow & \text{else} \end{cases}$$

is computable, as $f(\vec{x}) = 1 + \mu y[R(\vec{x}) \& y = y]$.

Theorem 9.2. *An n -ary relation $R \subseteq \mathbb{N}^n$ is partially decidable if and only if R is the domain of a computable partial function. In other words, there is a computable partial function $g : \mathbb{N}^n \rightarrow \mathbb{N}$ such that for all $\vec{x} \in \mathbb{N}^n$,*

$$R(\vec{x}) \iff g(\vec{x}) \downarrow.$$

Proof. If R is partially decidable, then its partial characteristic function is a computable partial function with domain R . Now suppose that R is the domain of $\varphi_e^{(n)}$. Then,

$$f(\vec{x}) = \begin{cases} 1 & R(\vec{x}) \\ \uparrow & \text{else} \end{cases}$$

is computable, since $f(\vec{x}) = 1 + 0 \cdot \mu y T_n(e, \vec{x}, y)$. ■

It follows that $W_0^{(n)}, W_1^{(n)}, W_2^{(n)}, \dots$ is an enumeration of all partially decidable n -ary relations.

Theorem 9.3. *An n -ary relation $R \subseteq \mathbb{N}^n$ is partially decidable if and only if there is a $(n+1)$ -ary computable relation $Q \subseteq \mathbb{N}^{n+1}$ such that, for any $\vec{x} \in \mathbb{N}^n$,*

$$R(\vec{x}) \iff \exists y Q(\vec{x}, y).$$

Proof. Suppose that R is partially decidable. Then $R = W_e^{(n)}$ for some $e \in \mathbb{N}$, so

$$R(\vec{x}) \iff \varphi_e^{(n)}(\vec{x}) \downarrow \iff \exists y T_n(e, \vec{x}, y),$$

where T_n is the Kleene T Predicate. Define $Q(\vec{x}, y) = T_n(e, \vec{x}, y)$. Now suppose that R is a relation satisfying

$$R(\vec{x}) \iff \exists y Q(\vec{x}, y)$$

for a decidable relation Q . Define $f(\vec{x}) = \mu y Q(\vec{x}, y)$. Then f is computable, and $R = \text{Dom}(f)$, so R is partially decidable. ■

Theorem 9.4. *If R and Q are partially decidable, then the relations $R \& Q$ and $R \vee Q$ are partially decidable. If R is a partially decidable $n+1$ -ary relation, the relations $\exists^< R(\vec{x}, y) \iff \exists (x < y) R(\vec{x}, z)$, $\forall^< R(\vec{x}, y) \iff (\forall z < y) R(\vec{x}, z)$ and $\exists R(\vec{x}) \iff \exists y R(\vec{x}, y)$.*

Proof. If $R = \text{Dom}(f)$ and $Q = \text{Dom}(g)$, then $R \& Q = \text{Dom}(f + g)$. Now, if $R(\vec{x}) \iff \exists y R'(\vec{x}, y)$ and $Q(\vec{x}) \iff \exists y Q'(\vec{x}, y)$, then $(R \vee Q)(\vec{x}) \iff \exists y (R'(\vec{x}, y) \vee Q'(\vec{x}, y))$. Now, suppose that R is partially decidable. Now let R be a partially decidable $n+1$ -ary relation. Take Q so that $R(\vec{x}, y) \iff \exists z Q(\vec{x}, y, z)$. Then, $\exists R(\vec{x}) \iff \exists y R(\vec{x}, y) \iff \exists y (\exists z Q(\vec{x}, y, z)) \iff \exists a Q(\vec{x}, (a)_0, (a)_1)$. We also have that $\exists^< R(\vec{x}, y) \iff \exists a [Q(\vec{x}, (a)_0, (a)_1) \& (a)_0 < y]$ and that $\forall^< R(\vec{x}, y) \iff \exists a [(\forall i < y)((a)_i = i) \& Q(\vec{x}, (a)_i, (a)_{i+y})]$. ■

By simple induction, if R is a partially decidable $n+k$ -ary relation, then the relation

$$Q(\vec{x}) \iff \exists x_1 \exists x_2 \cdots \exists x_k R(\vec{x}, x_1, \dots, x_k)$$

is partially decidable.

For example, for fixed n , the relation $R^{(n)}(e) \iff W_e^{(n)} \neq \emptyset$ is partially decidable, as $R^{(n)}(e) \iff \text{Dom}(\varphi_e^{(n)}) \neq \emptyset \iff \exists x \varphi_e^{(n)}(x) \downarrow$. By closure, this is partially decidable.

Theorem 9.5. *Let R be an n -ary relation. R is decidable if and only if R and $\neg R$ are both partially decidable.*

Proof. If R is decidable, then R is partially decidable and $\neg R$ is decidable, so $\neg R$ is partially decidable. Now suppose that R and $\neg R$ are partially decidable. Take e_0 and e_1 such that $R = W_{e_0}^{(n)}$ and $\neg R = W_{e_1}^{(n)}$. Define $f(\vec{x}) = \mu y [T_n(e_0, \vec{x}, y) \vee T_n(e_1, \vec{x}, y)]$. Note that exactly one of $R(\vec{x})$ and $\neg R(\vec{x})$ holds, so exactly one of $\varphi_{e_0}^{(n)}(\vec{x}) \downarrow$ or $\varphi_{e_1}^{(n)}(\vec{x})$. Thus, f is a computable total function, and $R(\vec{x}) \iff \varphi_{e_0}^{(n)}(\vec{x}) \downarrow \iff T_n(e_0, \vec{x}, f(\vec{x}))$. ■

Thus, if R is partially decidable but not decidable, then $\neg R$ is not partially decidable.

Theorem 9.6. *Let $f : \mathbb{N}^n \rightarrow \mathbb{N}$ be a partial function. Then f is computable if and only if its graph relation G_f is partially decidable.*

Proof. Suppose that f is computable, say $f = \varphi_e^{(n)}$. Then $f(\vec{x}) \downarrow = y \iff \exists z [T_n(e, \vec{x}, z) \& U(z) = y]$, which is partially decidable. If G_f is partially decidable. Pick a decidable relation R such that $G_f(\vec{x}, y) \iff \exists z R(\vec{x}, y, z)$. Then $f(\vec{x}) = (\mu a R(\vec{x}, (a)_0, (a)_1))_0$. This is computable, so f is computable. ■

10 Recursively Enumerable Sets

Recall that a unary relation on $\mathbb{N}$ is a subset of $\mathbb{N}$. Decidable unary relations on $\mathbb{N}$ are called computable or recursive sets. Now, if A and B are partially decidable, then the sets $A \cup B$ and $A \cap B$ are partially decidable (these are the relations $A \vee B$ and $A \& B$). Similarly, A is computable if and only if A and $\mathbb{N} \setminus A$ are partially decidable.

Notation. We write $K = \{x \in \mathbb{N} : \varphi_x(x) \downarrow\}$.

Definition 10.1. A set $A \subseteq \mathbb{N}$ is recursively enumerable or computably enumerable if either $A = \emptyset$ or if there exists a computable total function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $A = \text{Ran}(f)$. We say that f enumerates A , since $A = \{f(0), f(1), f(2), \dots\}$.

Theorem 10.2. *A set $A \subseteq \mathbb{N}$ is semi-recursive if and only if it is recursively enumerable.*

Proof. Suppose that A is recursively enumerable. Then there is a total computable function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $A = \text{Ran}(f)$. Then, $y \in A \iff \exists x, f(x) = y$. The relation $f(x) = y$ is the graph of a computable total function, and so is decidable. Thus, $\exists x, f(x) = y$ is semi-decidable. Now suppose that A is semi-recursive, nonempty and infinite. Pick a decidable relation $R \subseteq \mathbb{N}^2$ such that $x \in A \iff \exists y, R(x, y)$. Define $f(n) = (\mu z [z \geq n \& R((z)_0, (z)_1)])_0$. If $x \in A$ and $R(x, y)$, then $f(\langle x, y \rangle) = x$. Since A is infinite, f is total. ■

Theorem 10.3. *For a set $A \subseteq \mathbb{N}$, the following are equivalent.*

- (1) A is semi-computable.
- (2) $A = W_e$ for some $e \in \mathbb{N}$.

(3) There is a computable binary relation R such that, for every $x \in \mathbb{N}$

$$x \in A \iff \exists y R(x, y).$$

(4) There is a partially decidable $(n+1)$ -ary relation R such that, for every $x \in \mathbb{N}$

$$x \in A \iff \exists y_1, \dots, \exists y_n R(x, y_1, \dots, y_n).$$

(5) A is recursively enumerable.

(6) $A = \emptyset$ or A is the range of a computable injective total function.

(7) $A = E_e$ for some $e \in \mathbb{N}$.

Example 10.4. The set $A = \{x \in \mathbb{N} : \varphi_x \text{ is not total}\}$ is not recursively enumerable. We can computably reduce membership in K^c to membership in A . Define

$$g(x, y) = \begin{cases} 0 & \varphi_x(x) \downarrow \\ \uparrow & \text{else} \end{cases}.$$

This is computable. By the s-m-n theorem, there is a computable f such that

$$\varphi_{f(x)}(y) = g(x, y).$$

Now,

$$x \in K^c \iff \varphi_x(x) \uparrow \iff \forall y g(x, y) \uparrow \iff W_{f(x)} = \emptyset \iff \varphi_{f(x)} \text{ is not total}.$$

Thus, if A was semi-recursive, then K^c would be semi-recursive, which it is not.

Theorem 10.5. *Let $A \subseteq \mathbb{N}$ be infinite. Then A is recursive if and only if A is the range of a computable total function that is strictly increasing.*

Proof. Suppose that A is recursive. Let w_0 be the least element of A . Define

$$f(0) = w_0 \quad f(n+1) = \mu x [x \in A \& x > f(n)].$$

Then f is computable and enumerates A in increasing order. Now suppose that f is computable and enumerates A in increasing order. Since f is strictly increasing, $f(n) \geq n$. Write $G^f(x, y) \iff G_f(y, x)$. Then,

$$x \in A \iff \exists y [f(y) = x] \iff \exists y [y \leq x \& G^f(x, y)] \iff (\exists^< G^f)(x, x+1).$$

■

Theorem 10.6. *Every infinite recursively enumerable set has an infinite recursive subset.*

Proof. Let A be an infinite recursively enumerable set. Pick a total computable function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $A = \text{Ran}(f)$. Define $g : \mathbb{N} \rightarrow \mathbb{N}$ by $g(0) = f(0)$ and $g(n+1) = f(\mu x [f(x) > g(n)])$. Then g must be computable, total, strictly increasing, and $\text{Ran}(g) \subseteq \text{Ran}(f)$. ■

Theorem 10.7. *For a set $A \subseteq \mathbb{N}$, the following are equivalent:*

- (1) *A is semi-computable.*
- (2) *$A = W_e$ for some $e \in \mathbb{N}$.*
- (3) *There is a computable binary relation R such that for every $x \in \mathbb{N}$ $x \in A \iff \exists y R(x, y)$.*
- (4) *There is a partially decidable $n+1$ -ary relation R such that for every $x \in \mathbb{N}$ $x \in A \iff \exists y_1 \cdots \exists y_n R(x, y_1, \dots, y_n)$.*
- (5) *A is recursively enumerable.*
- (6) *A is finite or A is the range of a computable injective total function.*
- (7) *$A = E_e$ for some $e \in \mathbb{N}$.*

11 Many-One Reducibility

Definition 11.1. Let A and B be subsets of $\mathbb{N}$. We say that A is many-one reducible (or m -reducible) to B if there is a computable total function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that

$$x \in A \iff f(x) \in B.$$

Such an f is called an m -reduction from A to B .

We write $A \leq_m B$ to mean that A is m -reducible to B , and we write $f : A \leq_m B$ to mean that f is an m -reduction from A to B .

Note that if $A \leq_m B$ and B is recursive, then A is recursive, which is logically equivalent to A not being recursive implying that B is not recursive. The same is true for recursively enumerable sets.

Proposition 11.2. $\leq_m$ is a preorder on $\mathcal{P}(\mathbb{N})$. In other words, it is reflexive and transitive.

Proof. We have that $U_i^1 : A \leq_m A$ and that if $f : A \leq_m B$ and $g : B \leq_m C$, then $g \circ f$ is computable, and $x \in A \iff f(x) \in B \iff (g \circ f)(x) \in C$, so $g \circ f : A \leq_m C$. ■

Definition 11.3. A set A is m -complete if it is recursively enumerable and $B \leq_m A$ for all recursively enumerable sets B .

Theorem 11.4. $L = \{x : \varphi_x(x) \downarrow\}$ is m -complete.

Proof. Let B be recursively enumerable. Fix a recursive $R \subseteq \mathbb{N}^2$ such that, for all $x \in \mathbb{N}$, $x \in B \iff \exists y R(x, y)$. We want to find a computable f such that $x \in B \iff \varphi_{f(x)}(f(x)) \downarrow \iff f(x) \in K$. Define $g(x, y) = \mu z [R(x, z)]$. Then $g(x, y) \downarrow \iff x \in B$ for all y . By the s-m-n theorem, there is a computable $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $\varphi_{f(x)}(y) = g(x, y)$ for all x, y . Then $x \in B \iff g(x, y) \downarrow$ for all $y \in \mathbb{N}$. This occurs precisely when $g(x, f(x)) \downarrow$, so $\varphi_{f(x)}(f(x)) \downarrow$. Thus, $x \in B \iff \varphi_{f(x)}(f(x)) \downarrow$. ■

Proposition 11.5. *If A is m -complete, B is recursively enumerable, and $A \leq_m B$, then B is m -complete.*

Proof. For every recursively enumerable C , $C \leq_m A \leq_m B$, so $C \leq_m B$. ■

Definition 11.6. Sets A and B are said to be m -equivalent, denoted $A \equiv_m B$, if $A \leq_m B$ and $B \leq_m A$.

Proposition 11.7. $\equiv_m$ is an equivalence relation on $\mathcal{P}(\mathbb{N})$.

Proof. Reflexivity is immediate and symmetry follows from the definition. Transitivity follows from the transitivity of $\leq_m$. ■

We denote the $\equiv_m$ -equivalence class of A by

$$d_m(A) = \{B \in \mathcal{P}(\mathbb{N}) : A \equiv_m B\}.$$

We call $d_m(A)$ the degree of A .

If A and B are nontrivial recursive sets, then $A \equiv_m B$. Fix $b \in B$ and $c \notin B$. Define

$$f(x) = \begin{cases} b & x \in A \\ c & x \notin A \end{cases}.$$

Then f is computable, and $f : A \leq_m B$. We will usually write lowercase bold letters for m -degrees. Note that, for m -degrees $\mathbf{a}, \mathbf{b}$, $A \leq_m B$ for some $A \in \mathbf{a}, B \in \mathbf{b}$ if and only if the same is true for all $A \in \mathbf{a}$ and $B \in \mathbf{b}$.

Definition 11.8. For m -degrees $\mathbf{a}$ and $\mathbf{b}$, we write $\mathbf{a} \leq_m \mathbf{b}$ if $A \leq_m B$ for all (equivalently, some) $A \in \mathbf{a}$ and $B \in \mathbf{b}$.

We write $\mathbf{a} <_m \mathbf{b}$ if $\mathbf{a} \neq \mathbf{b}$ and $\mathbf{a} \leq_m \mathbf{b}$.

Proposition 11.9. $\leq_m$ is a partial ordering on the m -degrees. In other words, it is reflexive, transitive and antisymmetric.

Proof. The only nontrivial statement here is antisymmetry. Suppose that $\mathbf{a} \leq_m \mathbf{b}$ and $\mathbf{b} \leq_m \mathbf{a}$. Fix $A \in \mathbf{a}$ and $B \in \mathbf{b}$. Then $A \leq_m B$ and $B \leq_m A$, so $A \equiv_m B$. Thus, $\mathbf{a} = d_m(A) = d_m(B) = \mathbf{b}$. ■

We write $\mathbf{0} = d_m(\emptyset)$, $\mathbf{n} = d_m(\mathbb{N})$, and $\mathbf{0}_m = d_m(A)$, where A is any recursive set other than $\emptyset$ and $\mathbb{N}$. Finally, we denote $\mathbf{0}'_m = d_m(K)$. We call a degree $\mathbf{a}$ an *re m -degree* if it contains recursively enumerable set $A \in \mathbf{a}$.

Theorem 11.10. *For any two m -degrees, $\mathbf{a}, \mathbf{b}$, there is a least upper bound $\mathbf{c}$ such that $\mathbf{a} \leq_m \mathbf{c}$ and $\mathbf{b} \leq_m \mathbf{c}$, and there is not $\mathbf{d} < \mathbf{c}$ such that $\mathbf{a} \leq_m \mathbf{d}$ and $\mathbf{b} \leq_m \mathbf{d}$.*

Proof. Take $A \in \mathbf{a}$ and $B \in \mathbf{b}$. Let $C = \{2x : x \in A\} \cup \{2x + 1 : x \in B\} = A \oplus B$, and let $\mathbf{c} = d_m(C)$. Note that $x \in A \iff 2x \in A \oplus B$ and $x \in B \iff 2x + 1 \in A \oplus B$, so $f_0 : A \leq_m A \oplus B$ and $f_1 : B \leq_m A \oplus B$, where $f_0(x) = 2x$ and $f_1(x) = 2x + 1$. Thus, $\mathbf{c}$ is an upper bound for $\{\mathbf{a}, \mathbf{b}\}$. Suppose that $\mathbf{d} \geq_m \mathbf{a}, \mathbf{b}$. Take $D \in \mathbf{d}$ and $h_0 : A \leq_m D$, $h_1 : B \leq_m D$. Then $x = 2k \in A \oplus B \iff k \in A \iff h_0(k) \in D$ and $x = 2k + 1 \in A \oplus B \iff k \in B \iff h_1(k) \in D$, so $h : A \oplus B \leq_m D$ where $h(x) = \begin{cases} h_0(k) & x \text{ even} \\ h_1(k) & x \text{ odd} \end{cases}$. ■

We will prove the existence of a set A which is recursively enumerable, not recursive, and not m -complete. In other words, we will find an re m -degree $\mathbf{a}$ such that $\mathbf{0}_m <_m \mathbf{a} <_m \mathbf{0}'_m$.

We will first need to study other sets.

Definition 11.11. A set A is productive if there is a total computable function $p : \mathbb{N} \rightarrow \mathbb{N}$ such that, for any $e \in \mathbb{N}$,

$$W_e \subseteq A \implies p(e) \in A \setminus W_e.$$

The function p is called a productive function for A .

Note that if A is productive, it is not recursively enumerable, as if it were then there would be e such that $A = W_e$. Then we would have $p(e) \in A \setminus W_e = \emptyset$.

Proposition 11.12. K^c is productive.

Proof. Define $p(e) = e$ for all $e \in \mathbb{N}$. Suppose that $W_e \in K^c$ and that $e \in W_e$. Since $W_e \subseteq K^c$, $e \notin K$. Thus, $e \notin W_e$, a contradiction. Hence, $e \in K^c \setminus W_e$, as desired. ■

Proposition 11.13. If A is productive and $A \leq_m B$, then B is productive.

Proof. Let $p : \mathbb{N} \rightarrow \mathbb{N}$ be the productive function for A , and let $f : A \leq_m B$. We first show that there is a computable total $s : \mathbb{N} \rightarrow \mathbb{N}$ such that $W_{s(e)} = f^{-1}(W_e)$. Apply the s-m-n theorem to $g(e, x) = \begin{cases} 0 & f(x) \in W_e \\ \uparrow & \text{else} \end{cases}$. We claim that $q(e) = f(p(s(e)))$ is a productive function for B . Suppose that $W_e \subseteq B$. Then $W_{s(e)} = f^{-1}(W_e) \subseteq A$, so $p(s(e)) \in A \setminus W_{s(e)}$, meaning that $f(p(s(e))) \in B \setminus W_e$. ■

Note that if $K \leq_m A$, then $K^c \leq_m A^c$. Thus, for any m -complete set A , A^c is productive.

Definition 11.14. A set is creative if it is recursively enumerable and its complement is productive.

For example, A is creative.

Theorem 11.15. A set A is creative if and only if it is m -complete.

Proof. We have already shown that m -complete sets are creative. Suppose that A is creative, and let B be recursively enumerable. We want to show that $B \leq_m A$. Let p be the productive function for A^c . We claim that it suffices to find a computable total function

$$f : \mathbb{N} \rightarrow \mathbb{N} \text{ such that } W_{f(x)} = \begin{cases} \{p(f(x))\} & x \in B \\ \emptyset & x \notin B \end{cases}. \text{ Suppose that we have such an } f. \text{ Then}$$

$g(x) = p(f(x))$ is an m -reduction from B to A . We prove this in two cases. If $x \in B$, this means $W_{f(x)} = \{p(f(x))\}$. Suppose that $p(f(x)) \notin A$. Then $W_{f(x)} \subseteq A^c$, meaning $p(f(x)) \in A^c \setminus W_{f(x)}$, meaning that $p(f(x)) \neq p(f(x))$. Thus, $p(f(x)) \in A$. If $x \notin B$, then $W_{f(x)} = \emptyset \subseteq A^c$, so $p(f(x)) \in A^c \setminus W_{f(x)} = A^c$, meaning that $p(f(x)) \notin A$. We now construct

such an f . We want $g(x, y) = \varphi_{f(x)}(y) = \begin{cases} 0 & y = p(f(x)) \& x \in B \\ \uparrow & \text{else} \end{cases}$. Let S_1^1 be the function from the s-m-n theorem $\varphi_{s_1^1(e, x)}(y) = \varphi_e^{(2)}(x, y)$ for all $e, x, y \in \mathbb{N}$. Define

$$h(e, x, y) = \begin{cases} 0 & y = p(S_1^1(e, x)) \& x \in B \\ \uparrow & \text{else} \end{cases}.$$

By the second recursion theorem, there is e^* such that $\varphi_{S_1^1(e^*, x)}(y) = \varphi_{e^*}(x, y) = h(e^*, x, y)$.

Define $f(x) = S_1^1(e^*, x)$. Then, $\varphi_{f(x)}(y) = \varphi_{S_1^1(e^*, x)}(y) = \begin{cases} 0 & y = p(S_1^1(e^*, x)) \& x \in B \\ \uparrow & \text{else} \end{cases} = \begin{cases} 0 & y = p(f(x)) \& x \in B \\ \uparrow & \text{else} \end{cases}$. ■

Theorem 11.16. *If A is productive, then it contains an infinite recursively enumerable subset.*

Proof. Let A be productive, and let $p : \mathbb{N} \rightarrow \mathbb{N}$ be a productive function for A . We want to find a computable function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $W_{f(0)} = \emptyset$, $W_{f(1)} = W_{f(0)} \cup \{p(f(0))\}$, and in general that $W_{f(n+1)} = W_{f(n)} \cup \{p(f(n))\}$. With such an f , define $h(n) = p(f(n))$. Then h is computable, and $\text{Ran}(h)$ is an infinite recursively enumerable subset of A . Note that $h(n+1) = p(f(n+1)) \in A \setminus W_{f(n+1)}$, which means in particular that h is injective. Hence, h gives us an infinite recursively enumerable subset of A . We first show that there is computable $S : \mathbb{N} \rightarrow \mathbb{N}$ such that, for all x , $W_{S(x)} = W_x \cup \{p(x)\}$. We apply the s-m-n

theorem to $g(x, y) = \begin{cases} 0 & y = p(x) \vee y \in W_x \\ \uparrow & \text{else} \end{cases}$. Then define f by recursion: $f(0) = e_0$ where $W_{e_0} = \emptyset$, and $f(n+1) = S(f(n))$. Then $W_{f(n+1)} = W_{S(f(n))} = W_{f(n)} \cup \{p(f(n))\}$. f is computable. ■

Corollary 11.17. *If B is recursively enumerable but its complement does not contain an infinite recursively enumerable subset, then B is not creative.*

Definition 11.18. A set A is simple if it is recursively enumerable, A^c is infinite, and A^c does not contain any infinite recursively enumerable subsets.

Proposition 11.19. *A simple set A is not creative or recursive, and hence not m -complete. Thus, $\mathbf{0}_m <_m d_m(A) <_m \mathbf{0}'_m$.*

Proof. The set A^c does not contain an infinite recursively enumerable subset, so A is not creative. Now, A^c in particular cannot be recursively enumerable, as it is infinite, so A is not recursive. ■

Hence, the following theorem finishes our problem.

Theorem 11.20. *Simple sets exist.*

Proof. Define $R(e, x) \iff x \in W_e \& x > 2e$. Get a computable $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $f(e) \downarrow \iff \exists x R(e, x)$ and $f(e) \downarrow \implies R(e, f(e))$. Let $A = \text{Ran}(f)$. Since f is computable, A is recursively enumerable. If W_e is infinite, there is x such that $x \in W_e$ and $x > 2e$. Thus, there is x such that $R(e, x)$, meaning that $f(e) \downarrow \in A$ and $R(e, f(e))$. Then $f(e) \downarrow \in A \cap W_e$, so $A \cap W_e \neq \emptyset$. Thus, A^c cannot have an infinite recursively enumerable subset, as it would be W_e for some e , contradicting the previous statement. Finally, we show that A^c is infinite. Fix e such that $f(e) \downarrow$. Then $f(e) > 2e$. For all $e' > e$, $f(e') \downarrow \implies f(e') > 2e' > 2e$. Thus, $f^{-1}(\{0, 1, \dots, 2e\}) \subseteq \{0, 1, \dots, e-1\}$, so $|\text{Ran}(f) \cap \{0, 1, \dots, 2e\}| \leq e$. Then $|A^c \cap \{0, 1, \dots, 2e\}| \geq e+1$. Thus, $|A^c| \geq e+1$ for all e , so A^c is infinite. ■

12 Oracle Machines

Let $\alpha : \mathbb{N} \rightarrow \mathbb{N}$ be a total function, perhaps not computable. We consider an oracle machine with oracle α to be a machine that can compute computable functions, and $\alpha(k)$ for $k \in \mathbb{N}$. We formalize this. An unlimited register machine with oracle (URMO) is the same as our previous URM, except that we have an additional instruction $O(n)$ that replaces r_n with $\alpha(r_n)$.

Definition 12.1. For a positive integer n , oracle α and URMO program P , we define $f_P^{\alpha, n} : \mathbb{N}^n \rightarrow \mathbb{N}$ by running the URMO program P on the URMO with oracle α with the initial state of the input. If the machine halts, we define $f_P^{\alpha, n}$ to be the contents of R_1 .

Definition 12.2. An n -ary partial function $g : \mathbb{N}^n \rightarrow \mathbb{N}$ is called α -computable if $g = f_P^{\alpha, n}$ for some URMO program P .

Definition 12.3. The class of α -recursive partial functions is the smallest class of partial functions which contain the basic functions, contain α and are closed under substitution, recursion and minimalisation.

Theorem 12.4. A partial function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is α -computable if and only if it is α -recursive.

Definition 12.5. An n -ary relation $R \subseteq \mathbb{N}^n$ is called α -decidable or α -recursive if its characteristic function c_R is α -computable.

All structural properties of decidable relations hold for α -decidable relations as well. All our previous results work with α -computable functions (for instance, the Kleene T-predicate and the s-m-n theorem).

Definition 12.6. A set $A \subseteq \mathbb{N}^n$ is α -partially decidable if its partial characteristic function $c_R^p : \mathbb{N}^n \rightarrow \mathbb{N}$ is α -computable.

A set $A \subseteq \mathbb{N}$ which is α -partially decidable is said to be α -recursively enumerable.

Definition 12.7. For a set $A \subseteq \mathbb{N}$, we define A -computability to mean computability relative to the oracle c_A , where c_A is the characteristic of A . We write C^A for the set of A -computable functions.

13 Turing Reducibility

Definition 13.1. Let A and B be subsets of $\mathbb{N}$. We say A is Turing reducible to B , denoted $A \leq_T B$ if A is B -recursive. We say A and B are Turing equivalent, denoted $A \equiv_T B$ if $A \leq_T B$ and $B \leq_T A$.

For example, if A is recursive, then $A \leq_T B$ for any B . Note also that if $A \leq_m B$ then $A \leq_T B$: if $f : A \leq_m B$, then $x \in A \iff f(x) \in B$, and the latter is B -computable. An application of this is that $A \leq_T K$ for all recursively enumerable A , as K is m -complete.

Definition 13.2. We call a set B T -complete if B is recursively enumerable and $A \leq_T B$ for all recursively enumerable sets A .

Example 13.3. For any set A , $A \equiv_T A^c$.

Proposition 13.4. $\leq_T$ is a preordering on $\mathcal{P}(\mathbb{N})$, and $\equiv_T$ is an equivalence relation on $\mathcal{P}(\mathbb{N})$.

Proof. Reflexivity is trivial. For transitivity, we have that if $A \leq_T B \leq_T C$, then $C^A \subseteq C^B \subseteq C^C$, so $C^A \subseteq C^C$. Hence, $A \leq_T C$. That $\equiv_T$ is an equivalence relation follows immediately. ■

Definition 13.5. For a set $A \subseteq \mathbb{N}$, we write

$$d_T(A) = \{B \subseteq \mathbb{N} : A \equiv_T B\}.$$

We call this the Turing degree of A .

We denote by $\mathbf{0}$ the degree of recursive sets.

Definition 13.6. A Turing degree $\mathbf{a}$ is called an re Turing degree if there is a recursively enumerable set $A \in \mathbf{a}$.

We write $\mathbf{0}' = d_T(K)$. Note that $\mathbf{0}'$ is an re degree, but contains non recursively enumerable sets, such as K^c .

Definition 13.7. For Turing degrees $\mathbf{a}, \mathbf{b}$, we write $\mathbf{a} \leq_T \mathbf{b}$ if, for some (equivalently, all) $A \in \mathbf{a}$ and $B \in \mathbf{b}$, we have $A \leq_T B$.

For example, for any re degree $\mathbf{a}$, $\mathbf{a} \leq_T \mathbf{0}'$.

We write

$$K^A = \{x \in \mathbb{N} : \varphi_x^A(x) \downarrow\}.$$

Similarly, for any A , K^A is recursively enumerable with respect to A , and not A -decidable.

Proposition 13.8. K^A is T -complete with respect to A -recursively enumerable sets. In other words, $B \leq_T K^A$ for all A -recursively enumerable sets B .

Proof. Suppose that B is A -recursively enumerable. Then $B = \text{Dom}(f)$ for some $f \in C^A$. Define $g(x, y) = f(x)$. By the s-m-n theorem, there is $s : \mathbb{N} \rightarrow \mathbb{N}$ that is computable such that $\varphi_{s(x)}^A(y) = g(x, y) = f(x)$ for all x, y . Then $x \in B$ if and only if $f(x) \downarrow$ if and only if $\varphi_{s(x)}^A(s(x))$ converges, meaning that $s(x) \in K^A$. Since $B \leq_m K^A$, $B \leq_T K^A$. ■

Proposition 13.9. *For any sets A and B , if $A \leq_T B$ then $K^A \leq_T K^B$ and if $A \equiv_T B$ then $K^A \equiv_T K^B$.*

Proof. Suppose that $A \leq_T B$. Then $C^A \subseteq C^B$. Since K^A is A -recursively enumerable, $K^A = \text{Dom}(f)$ for some $f \in C^A \subseteq C^B$. Then K^A is also B -recursively enumerable. Hence, $K^A \leq_T K^B$ by the previous result. ■

Definition 13.10. Let $\mathbf{a}$ be a Turing degree. The Turing jump of A is

$$\mathbf{a}' = d_T(K^A) \quad A \in \mathbf{a}.$$

This is well defined by previous results.

Proposition 13.11. *For any Turing degrees $\mathbf{a}$ and $\mathbf{b}$,*

- $\mathbf{a} <_T \mathbf{a}'$
- if $\mathbf{a} \leq_T \mathbf{b}$ then $\mathbf{a}' \leq_T \mathbf{b}'$
- $\mathbf{0}' \leq_T \mathbf{a}'$
- if $B \in \mathbf{b}$ and $A \in \mathbf{a}$ and B is A -recursively enumerable, then $\mathbf{b} \leq_T \mathbf{a}'$.

Theorem 13.12. *Every recursively enumerable Turing degree $\mathbf{a} \neq \mathbf{0}$ contains a simple set.*

Proof. Let B be recursively enumerable, not recursive, and let f be a total injective computable function such that $B = \text{Ran}(f)$. Define

$$A = \{x \in \mathbb{N} : \exists y(y > x \& f(y) < f(x))\}.$$

Then A is simple. We show $B \equiv_T A$. Note that

$$\begin{aligned} x \in A &\iff \exists y(y > x \& f(y) < f(x)) \iff \exists z(z \in B \& z < f(z) \& f^{-1}(z) > x) \\ &\iff (\exists z < f(x))(z \in B \& f^{-1}(z) \notin \{0, 1, \dots, x\}) \\ &\iff (\exists z < f(x))(z \in B \& f(0) \neq z \& \dots \& f(x) \neq z). \end{aligned}$$

Since f is total and computable, its graph is recursive. The expression on the right is B -recursive. Hence, A is B -recursive. Now, take $n \notin A$ such that $f(n) > z$. Then.

$$z \in B \iff (\exists x < n)(f(x) = z).$$

Thus if n is A -computable, we are done. Define $g(z) = \mu n[n \notin A \& f(n) > z]$. This is A -computable, and total as A^c is infinite. ■

Corollary 13.13. *There exists a recursively enumerable set $A \in \mathbf{0} \setminus \mathbf{0}'_m$.*

Proof. there is $A \in \mathbf{0}'$ such that A is simple, but simple sets are not m -complete. ■

13.1 Incomparable Degrees

In this section, we consider finite sequences $(x_0, \dots, x_{n-1})$ as a partial function $f : \mathbb{N} \rightarrow \mathbb{N}$ such that $\text{Dom}(f) = \{1, \dots, n-1\}$ and $f(i) = x_i$. In particular, we write $(x_0, \dots, x_{n-1}) \sqsubseteq (y_0, \dots, y_{m-1})$ if $n \leq m$ and $x_i = y_i$ for $0 \leq i \leq n$. Consider $A \subseteq \mathbb{N}$. A finite sequence $\sigma = (x_0, \dots, x_{n-1})$ is called a finite approximation to A if, for all $k < n$ $k \in A \implies x_k = 1$ & $k \notin A \implies x_k = 0$. Thinking of σ as a partial function, this is saying $\sigma \sqsubseteq c_A$. We write $\sigma \sqsubseteq A$ in this case. For $s \in \mathbb{N}$, we write $A|_s$ for the finite approximation of A of length s .

Consider $\sigma_0 \sqsubset \sigma_1 \sqsubset \sigma_2 \sqsubset \dots$, where σ_n is a finite set approximation. Then they limit to a function $\alpha : \mathbb{N} \rightarrow \{0, 1\}$ with $\alpha(x) = (\lim_n \sigma_n)(x) = \sigma_m(x)$ for any m such that $\sigma_m(x)$ is defined.

We can consider using σ as an oracle, where σ is a finite set approximation. In this case, we add the condition that if the program asks for $\sigma(k)$ when σ is not defined at k , we do not halt.

Proposition 13.14. *Let σ and τ be finite set approximations. Then*

$$\varphi_e^\alpha(x) \downarrow \& \sigma \sqsubseteq \tau \implies \varphi_e^\tau(x) \downarrow = \varphi_e^\sigma(x) \downarrow.$$

Also, for any A and $e, x \in \mathbb{N}$, if $\varphi_e^A(x) \downarrow$ then there exists $s \in \mathbb{N}$ such that $\varphi_e^{A|_t}(x) \downarrow = \varphi_e^A(x)$ for all $t \geq s$.

Theorem 13.15 (Kleene-Post). *There are incomperable Turing degrees. In other words, there are A and B such that $A \not\leq_T B$ and $B \not\leq_T A$.*

Proof. We will define finite set approximations σ_n and τ_n , and define $A = \lim \sigma_n$ and $B = \lim \tau_n$. At stage $s+1$, we pick σ_{s+1} and τ_{s+1} to satisfy $c_A \neq \varphi_e^B$ if $s = 2e$, and $c_B \neq \varphi_e^A$ if $s = 2e+1$. However, this does not work fully, as we are constructing A and B . We break this down. In the first stage, we have that $\sigma_0 = \tau_0 = \emptyset$. If $s = 2e$, and we know σ_s and τ_s , we want to ensure that $c_A \neq \varphi_e^B$. Take x to be the smallest value such that $\sigma_s(x)$ is not defined. We want σ_{s+1} such that $\sigma_{s+1}(x) \neq \varphi_e^B(x)$. Take $\tau \sqsupseteq \tau_s$ such that $\varphi_e^\tau(x) \downarrow$. If this exists, set $\tau_{s+1} = \tau$ and $\sigma_{s+1}(y) = \begin{cases} \sigma_s(y) & y < x \\ 1 - \varphi_e^{\tau_{s+1}}(x) & y = x \end{cases}$. Then, whatever A and B are, $\tau_{s+1} \sqsubseteq B$,

meaning $\varphi_e^B(x) \downarrow$, and $\sigma_{s+1} \sqsubseteq A$, which means that $c_A(x) = \sigma_{s+1}(x) = 1 - \varphi_e^B(x)$, so that our requirement is fulfilled. If we cannot find such a τ , then no matter what $B \sqsupseteq \tau_s$ is, $\varphi_e^B(x) \uparrow$. Then set $\tau_{s+1} = \tau_s$ and $\sigma_{s+1}(y) = \begin{cases} \sigma_s(y) & y < x \\ 0 & y = x \end{cases}$. Then our condition is satisfied.

In the case $s = 2e+1$, we use the same construction, swapping the roles of A and B . By construction, c_A is not B -computable, and c_B is not A -computable. $\blacksquare$

Given a sequence of sets $A_0, A_1, \dots$, define

$$\bigoplus_n A_n = \{(n, x) : x \in A_n\}.$$

Notice that $A_m \leq_T \bigoplus_n A_n$, since $x \in A_m \iff (m, x) \in \bigoplus_n A_n$. We define the ω -jump of $\emptyset$ as

$$\emptyset^{(\omega)} = \bigoplus_n \emptyset^{(n)},$$

where $\emptyset^{(n)}$ denotes the n -th jump of $\emptyset$.

Definition 13.16. A sequence $A_0, A_1, \dots$ is called recursively independent if, for all m ,

$$A_m \not\leq_T \bigoplus_{n \neq m} A_n.$$

Theorem 13.17. *There exists a recursively independent sequence.*

14 Recursive Operators

For $n \in \mathbb{Z}^+$, let $\mathcal{F}_n$ be the set of all partial functions $f : \mathbb{N}^n \rightarrow \mathbb{N}$. An operator on partial functions is a function of the form $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$. For example, $\Phi : \mathcal{F}_1 \rightarrow \mathcal{F}_1$ by $\Phi(f) = 2f$, or $\Phi : \mathcal{F}_2 \rightarrow \mathcal{F}_1$ by $\Phi(f)(x) = f(x, x)$.

We will call a partial function finite if it has finite domain.

For a partial function $\theta : \mathbb{N}^n \rightarrow \mathbb{N}$ with $\text{Dom}(\theta)$ finite, we define the code of θ to be

$$\tilde{\theta} = \prod_{\vec{x} \in \text{Dom}(\theta)} p_{\langle \vec{x} \rangle}^{\theta(\vec{x})+1}$$

if $\text{Dom}(\theta) \neq \emptyset$. We define $\tilde{\theta} = 0$ if $\text{Dom}(\theta) = \emptyset$.

The following are decidable.

$$\begin{aligned} \text{FinPF}(u, n) &\iff u \text{ is a code for an } n\text{-ary finite partial function} \\ &\iff u \neq 1 \& (\forall v < u)[p_v | u \rightarrow \text{Seq}(v) \& \text{lh}(v) = n] \\ \text{Domain}_n(u, \vec{x}) &\iff u = \tilde{\theta}, \theta : \mathbb{N}^n \rightarrow \mathbb{N} \text{ finite, and } \vec{x} \in \text{Dom}(\theta) \\ &\iff \text{FinPF}(u, n) \& p_{\langle \vec{x} \rangle} | u \\ \text{Value}_n(u, \vec{x}, y) &\iff u = \tilde{\theta}, \theta : \mathbb{N}^n \rightarrow \mathbb{N} \text{ finite, } \vec{x} \in \text{Dom}(\theta), \text{ and } \theta(\vec{x}) = y \\ &\iff \text{Domain}_n(u, \vec{x}) \& p_{\langle \vec{x} \rangle}^{y+1} | u \& p_{\langle \vec{x} \rangle}^{y+2} \nmid u \end{aligned}$$

We also define $\text{Eval}_n : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ by

$$\text{Eval}_n(u, \vec{x}) = \begin{cases} \theta(\vec{x}) & u = \tilde{\theta}, \theta : \mathbb{N}^n \rightarrow \mathbb{N} \\ \uparrow & \text{else} \end{cases}.$$

This is computable and has computable domain.

For a partial function f , we write $\theta \sqsubseteq_{\text{fin}} f$ to mean that f extends θ and θ is finite.

Definition 14.1. An operator $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ is a recursive operator if there is a computable partial function $\varphi : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ such that, for any $f \in \mathcal{F}_m$, $\vec{x} \in \mathbb{N}^n$, $y \in \mathbb{N}$,

$$\Phi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \varphi(\tilde{\theta}, \vec{x}) = y.$$

For example, $\Phi : \mathcal{F}_1 \rightarrow \mathcal{F}_1$ by $\Phi(f) = 2f$ is recursive using $\varphi : \mathbb{N}^n \rightarrow \mathbb{N}$ by $\varphi(x, z) = 2 \text{Eval}_1(z, x)$. Then φ is computable.

Definition 14.2. Let $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ be an operator. We say that Φ is continuous if, for any $f \in \mathcal{F}_m$, $\vec{x} \in \mathbb{N}^n$ and $y \in \mathbb{N}$,

$$\Phi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \Phi(\theta)(\vec{x}) = y.$$

We say that Φ is monotone if, for any $f, g \in \mathcal{F}_m$,

$$f \sqsubseteq g \implies \Phi(f) \sqsubseteq \Phi(g).$$

Proposition 14.3. If $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ is recursive, then Φ is continuous and monotone.

Proof. Since Φ is recursive, there is $\varphi : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ computable such that for any $f \in \mathcal{F}_m$, $\vec{x} \in \mathbb{N}^n$, $y \in \mathbb{N}$, $\Phi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \varphi(\vec{\theta}, \vec{x}) = y$. Then, there is $\tau \sqsubseteq_{\text{fin}} \theta$ such that $\varphi(\vec{\tau}, \vec{x}) = y$ (take $\tau = \theta$). This means that $\Phi(\theta)(\vec{x}) = y$. Now, if there exists such a θ , then there is $\tau \sqsubseteq_{\text{fin}} \theta \sqsubseteq_{\text{fin}} f$ such that $\varphi(\vec{\tau}, \vec{x}) = y$ whenever $\Phi(\theta)(\vec{x}) = y$. Since $\tau \sqsubseteq_{\text{fin}} f$, we have that $\Phi(f)(\vec{x}) = y$. Now, $f \sqsubseteq g$ means that if $\Phi(f)(\vec{x}) = y$ then there is $\theta \sqsubseteq_{\text{fin}} f$ with $\varphi(\vec{\theta}, \vec{x}) = y$. Since $\theta \sqsubseteq_{\text{fin}} g$, we have that $\varphi(\vec{\theta}, \vec{x}) = y$ means $\Phi(g)(\vec{x}) = y$. ■

Theorem 14.4. An operator $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ is recursive if and only if Φ is continuous and the function $\varphi : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ defined by

$$\varphi(z, \vec{x}) = \begin{cases} \Phi(\theta)(\vec{x}) & z = \vec{\theta}, \theta \text{ finite} \\ \uparrow & \text{else} \end{cases}$$

is computable.

Proof. Suppose that Φ is continuous and φ is computable. Then for any $f, \vec{x}, y$, $\Phi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \Phi(\theta)(\vec{x}) = y$. This happens if and only if $(\exists \theta \sqsubseteq_{\text{fin}} f) \varphi(\vec{\theta}, \vec{x}) = y$. Since φ is computable, Φ is recursive. Now, if Φ is recursive, then it is continuous, and there is a computable ψ such that $\Phi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \psi(\vec{\theta}, \vec{x}) = y$. Now,

$$\begin{aligned} \varphi(z, \vec{x}) = y &\iff \text{FinPF}(z, m) \& \Phi(\theta)(\vec{x}) = y \iff \text{FinPF}(z, m) \& (\exists \tau \sqsubseteq_{\text{fin}} \theta) \psi(\vec{\tau}, \vec{x}) = y \\ &\iff (\exists u \leq z) [\text{FinPF}(u, m) \& "u \sqsubseteq z" \& \psi(u, \vec{x}) = y]. \end{aligned}$$

■

For example, if we define $\Phi : \mathcal{F}_{n+1} \rightarrow \mathcal{F}_n$ by

$$\Phi(f)(\vec{x}) = \mu y [f(\vec{x}, y) = 0],$$

then this is recursive. It is continuous, as if $\Phi(f)(\vec{a}) = w$, we can take

$$\theta(\vec{x}, y) = \begin{cases} f(\vec{x}, y) & \vec{x} = \vec{a} \& y \leq w \\ \uparrow & \text{else} \end{cases}.$$

Then $\theta \sqsubseteq_{\text{fin}} f$ and $\Phi(\theta)(\vec{a}) = w$, since $\theta(\vec{a}, k) \downarrow \neq 0$ for every $k < w$. Clearly the function φ is computable: $\varphi(z, \vec{x}) = \mu y [\text{Eval}_{n+1}(z, \vec{x}, y) = 0]$.

15 Effective Operations

For each positive integer n , let $\mathcal{C}^{(n)}$ be the set of n -ary computable partial functions. A function $\Psi : \mathcal{C}^{(m)} \rightarrow \mathcal{C}^{(n)}$ is called an effective operation on computable partial functions if there is a computable total function $h : \mathbb{N} \rightarrow \mathbb{N}$ such that

$$\Psi(\varphi_e^{(m)}) = \varphi_{h(e)}^{(n)}.$$

For example, if we fix $x_0 \in \mathbb{N}$, then $\Psi : \mathcal{C}^{(2)} \rightarrow \mathcal{C}^{(1)}$ given by $\Psi(f)(y) = f(x_0, y)$ is an effective operation on computable partial functions, as we may take h from the s-m-n theorem to be $h(e) = s_1^1(e, x_0)$.

Note that if $\Phi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ is recursive, then $\Phi(f)$ is computable whenever f is computable. Thus we can restrict it to $\Phi_e : \mathcal{C}^{(m)} \rightarrow \mathcal{C}^{(n)}$.

Lemma 15.1. *For any positive integer n , the relation*

$$\text{Extend}_n(e, z) \iff z = \tilde{\theta} \text{ for } n\text{-ary finite } \theta, \text{ and } \theta \sqsubseteq_{\text{fin}} \varphi_e^{(n)}.$$

Proof. We have that

$$\text{Extend}_n(e, z) \iff \text{FinPF}(z, n) \& (\forall x < z) [\text{Domain}_n(z, x) \rightarrow \text{Eval}_n(z, x) = \varphi_e^{(n)}(x)].$$

■

Theorem 15.2. *Let $\Psi : \mathcal{F}_m \rightarrow \mathcal{F}_n$ be a recursive operator. Then its restriction to computable partial functions is an effective operation.*

Proof. Pick computable ψ such that

$$\Psi(f)(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \psi(\tilde{\theta}, \vec{x}) = y.$$

Define $\alpha : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ by

$$\alpha(e, \vec{x}) = \Psi(\varphi_e^{(m)})(\vec{x}).$$

If we show that α is computable, then by the s-m-n theorem, there is a computable h such that

$$\varphi_{h(e)}^{(n)}(\vec{x}) = \alpha(e, \vec{x}) = \Psi(\varphi_e^{(m)})(\vec{x}).$$

Now,

$$\alpha(e, \vec{x}) = y \iff \Psi(\varphi_e^{(m)})(\vec{x}) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} \varphi_e^{(m)}) \psi(\tilde{\theta}, \vec{x}) = y \iff (\exists z) (\text{Extend}_m(e, z) \& \psi(z, \vec{x}) = y).$$

■

Theorem 15.3. *Let $\mathcal{A} \subseteq \mathcal{C}^{(1)}$ be a collection of computable partial functions, and let $A = \{e : \varphi_e \in \mathcal{A}\}$ be its index set. If A is recursively enumerable, then $f \in \mathcal{A}$ if and only if there exists a finite $\theta \sqsubseteq_{\text{fin}} f$ such that $\theta \in \mathcal{A}$.*

Proof. Suppose that $f \in \mathcal{A}$. Assume that $\theta \notin \mathcal{A}$ for all $\theta \sqsubseteq_{\text{fin}} f$. We reduce $K^c \leq_m A$, which contradicts that A is recursively enumerable. Define

$$g(e, x) = \begin{cases} f(x) & (\forall s < x) \neg T(e, e, s) \\ \uparrow & \text{else} \end{cases}.$$

By the s-m-n theorem, there is a computable s such that $\varphi_{s(e)}(x) = g(e, x)$. Now,

$$\begin{aligned} e \in K^c &\implies \varphi_e(e) \uparrow \implies (\forall x)(\forall s < x) \neg T(e, e, s) \implies (\forall x)g(e, x) \simeq f(x) \\ &\implies \varphi_{s(e)} = f \implies \varphi_{s(e)} \in \mathcal{A} \implies s(e) \in A \end{aligned}$$

and

$$\begin{aligned} e \in K &\implies \varphi_e(e) \downarrow \implies \exists y_0 T_1(e, e, y_0) \implies (\forall x > y_0) g(e, x) \uparrow \& (\forall x \leq y_0) g(e, x) \simeq f(x) \\ &\implies \varphi_{s(e)} \sqsubseteq_{\text{fin}} f \implies \varphi_{s(e)} \notin \mathcal{A} \implies s(e) \notin A. \end{aligned}$$

Hence, $s : K^c \leq_m A$. Now suppose that $\theta \sqsubseteq_{\text{fin}} f$, $\theta \in \mathcal{A}$ and $f \notin \mathcal{A}$. We show again that $K^c \leq_m A$. Define

$$h(e, x) = \begin{cases} f(x) & \theta(x) \downarrow \vee x \in K \\ \uparrow & \text{else} \end{cases}.$$

Take u such that $\varphi_{u(e)}(x) = h(e, x)$. Now,

$$e \in K^c \implies (h(e, x) \downarrow \iff \theta(x) \downarrow) \implies \varphi_{u(e)} = \theta \implies \varphi_{u(e)} \in \mathcal{A} \implies u(e) \in A$$

and

$$e \in K \implies (h(e, x) \downarrow \iff f(x) \downarrow) \implies \varphi_{u(e)} = f \implies \varphi_{u(e)} \notin \mathcal{A} \implies u(e) \notin A.$$

Thus $u : K^c \leq_m A$. ■

Theorem 15.4. *If $\Psi_0 : \mathcal{C}^{(1)} \rightarrow \mathcal{C}^{(1)}$ is an effective operation on computable partial functions, then there exists a recursive operator $\Psi : \mathcal{F}_1 \rightarrow \mathcal{F}_1$ such that $\Psi(f) = \Psi_0(f)$ for all $f \in \mathcal{C}^{(1)}$.*

Proof. Since Ψ_0 is effective, we can pick a total computable $h : \mathbb{N} \rightarrow \mathbb{N}$ such that $\Psi_0(\varphi_e) = \varphi_{h(e)}$ for all $e \in \mathbb{N}$. Define

$$\Psi(f)(x) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \Psi_0(\theta)(x) = y.$$

We have to show that Ψ is well defined, extends Ψ_0 and is recursive. We first show that

$$\Psi_0(f)(x) = y \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \Psi_0(\theta)(x) = y.$$

Fix x, y , and let $A = \{e : \Psi_0(\varphi_e)(x) = y\}$. Then A is an index set, and A is recursively enumerable, as $e \in A \iff \Psi_0(\varphi_e)(x) = y \iff \varphi_{h(e)}(x) = y$. Let $\mathcal{A} = \{f \in \mathcal{C}^{(1)} : \Psi_0(f)(x) = y\}$. Then $f \in \mathcal{A} \iff (\exists \theta \sqsubseteq_{\text{fin}} f) \theta \in \mathcal{A}$, which means that $\Psi_0(f)(x) = y$ if and

only if there is $\theta \sqsubseteq_{\text{fin}} f$ such that $\Psi_0(\theta)(x) = y$. Now suppose that $f \in \mathcal{F}_1$ and there exist $\theta_i \sqsubseteq_{\text{fin}} f$, and that $\Psi_0(\theta_i)(x) = y_0$ for $i = 1, 2$. We must show that $y_1 = y_2$. Pick θ such that $\theta_i \sqsubseteq \theta \sqsubseteq f$. We have that $\Psi_0(\theta)(x) = \Psi_0(\theta_i)(x) = y_i$, so we are done. Now, for any $f \in \mathcal{C}^{(1)}$, we have that

$$\Psi_0(f)(x) = y \iff \exists \theta \sqsubseteq_{\text{fin}} f \Psi_0(\theta)(x) = y \iff \Psi(f)(x) = y.$$

It remains to show that Ψ is recursive. We first show that there is a computable total function $c : \mathbb{N} \rightarrow \mathbb{N}$ such that, for any finite $\theta : \mathbb{N} \rightarrow \mathbb{N}$, $\varphi_{c(\tilde{\theta})} = \theta$. Let $\alpha(z, x) = \text{Eval}_1(z, x)$, and apply the s-m-n theorem to get a computable c such that $\varphi_{c(z)}(x) = \alpha(z, x)$ for all z, x . If $z = \tilde{\theta}$, then this becomes $\varphi_{c(\tilde{\theta})}(x) = \alpha(\tilde{\theta}, x) = \text{Eval}_1(\tilde{\theta}, x) = \theta(x)$. Finally, we show Ψ is recursive. It is enough to show that Ψ is continuous and that

$$\psi(z, x) = \begin{cases} \Psi(\theta)(x) & \exists \theta : \mathbb{N} \rightarrow \mathbb{N}, z = \tilde{\theta} \\ \uparrow & \text{else} \end{cases}.$$

By definition, Ψ is continuous, and for $z = \tilde{\theta}$,

$$\psi(z, x) = \Psi(\theta)(x) = \Psi(\varphi_{c(\tilde{\theta})})(x) = \Psi_0(\varphi_{c(z)})(z) = \varphi_{h(c(z))}(x).$$

■

The same theorem holds for n .

Theorem 15.5 (The First Recursion Theorem). *Let $\Psi : \mathcal{F}_m \rightarrow \mathcal{F}_m$ be a recursive operator. Then there exists a computable partial function $f_\Psi : \mathbb{N}^m \rightarrow \mathbb{N}^m$ which is the least fixed point of Ψ . In other words, $\Psi(f_\Psi) = f_\Psi$, and for any $g \in \mathcal{F}_m$, $\Psi(g) = g$ means that $f_\Psi \sqsubseteq g$. In particular, if f_Ψ is total, it is the only fixed point of Ψ .*

Proof. We show this for $m = 1$. Since Ψ is recursive, it is continuous and monotone. We define a sequence $f_0, f_1, f_2, \dots$ by $f_0 = \varepsilon$ (the empty function), and $f_{n+1} = \Psi(f_n)$. By monotonicity, $f_0 \sqsubseteq f_1$, meaning that $f_1 = \Psi(f_0) \sqsubseteq \Psi(f_1) = f_2$, and induction implies that $f_0 \sqsubseteq f_1 \sqsubseteq f_2 \sqsubseteq \dots$. Set $f_\Psi = \lim_{n \rightarrow \infty} f_n$. We claim that f_Ψ is a fixed point. We first show that $f_\Psi \sqsubseteq \Psi(f_\Psi)$. Suppose that $f_\Psi(x) = y$. Then there is n such that $f_n(x) = y$. As $f_n \sqsubseteq f_\Psi$, $\Psi(f_n) \sqsubseteq \Psi(f_\Psi)$, meaning that $f_n \sqsubseteq f_{n+1} \sqsubseteq \Psi(f_\Psi)$. As $f_n(x) = y$, $\Psi(f_\Psi)(x) = y$. Now we show that $\Psi(f_\Psi) \sqsubseteq f_\Psi$. Suppose that $\Psi(f_\Psi)(x) = y$. By continuity, there is $\theta \sqsubseteq_{\text{fin}} f_\Psi$ such that $\Psi(\theta)(x) = y$. Pick n large enough so that $\theta \sqsubseteq f_n$. Then $\Psi(\theta) \sqsubseteq \Psi(f_n) = f_{n+1}$. Since $\Psi(\theta)(x) = y$, $f_{n+1}(x) = y$, so $f_\Psi(x) = y$. Now, suppose that $\Psi(g) = g$. Clearly $f_0 \sqsubseteq g$, and so $\Psi(f_0) = f_1 \sqsubseteq g = \Psi(g)$. Continuing, $f_n \sqsubseteq g$ for each n . Then, $f_\Psi = \lim_{n \rightarrow \infty} f_n \sqsubseteq g$. Finally, we show that f_Ψ is computable. There is a computable h such that $\Psi(\varphi_e) = \varphi_{h(e)}$ for all e . Define u by $u(0) = e_0$ for some fixed e_0 that is an index for ε , and define $u(n+1) = h(u(n))$. Then u is computable, and $f_n = \varphi_{u(n)}$ for all n . Now,

$$f_\Psi(x) = y \iff \exists n \varphi_{u(n)}(x) = y,$$

so the graph of f is computable. ■